



[Harman] 세미콘 아카데미 4주차_과제

레포트

Design Verification Project (linebuffer , rgboffset)

[14조] 김준회(팀장), 이승후, 윤호승, 조현우

Contents

1. Overview

2. DUT Spec Analysis

3. Test Planning

4. TB 작성

5. 시나리오별 시뮬레이션 결과

6. 시나리오별 Report 결과

7. Troubleshooting

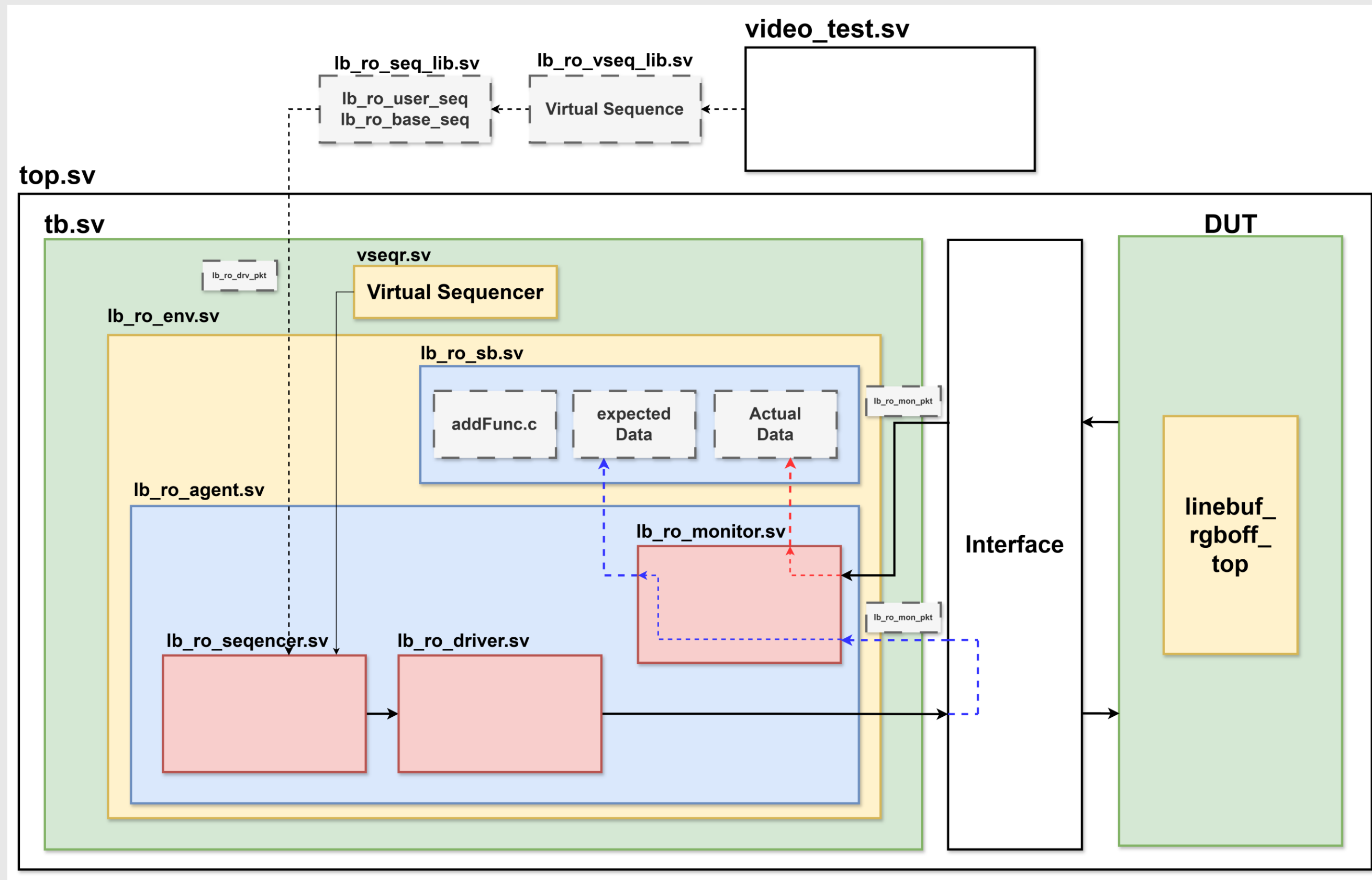


1. Overview

01

Overview

Testbench Architecture

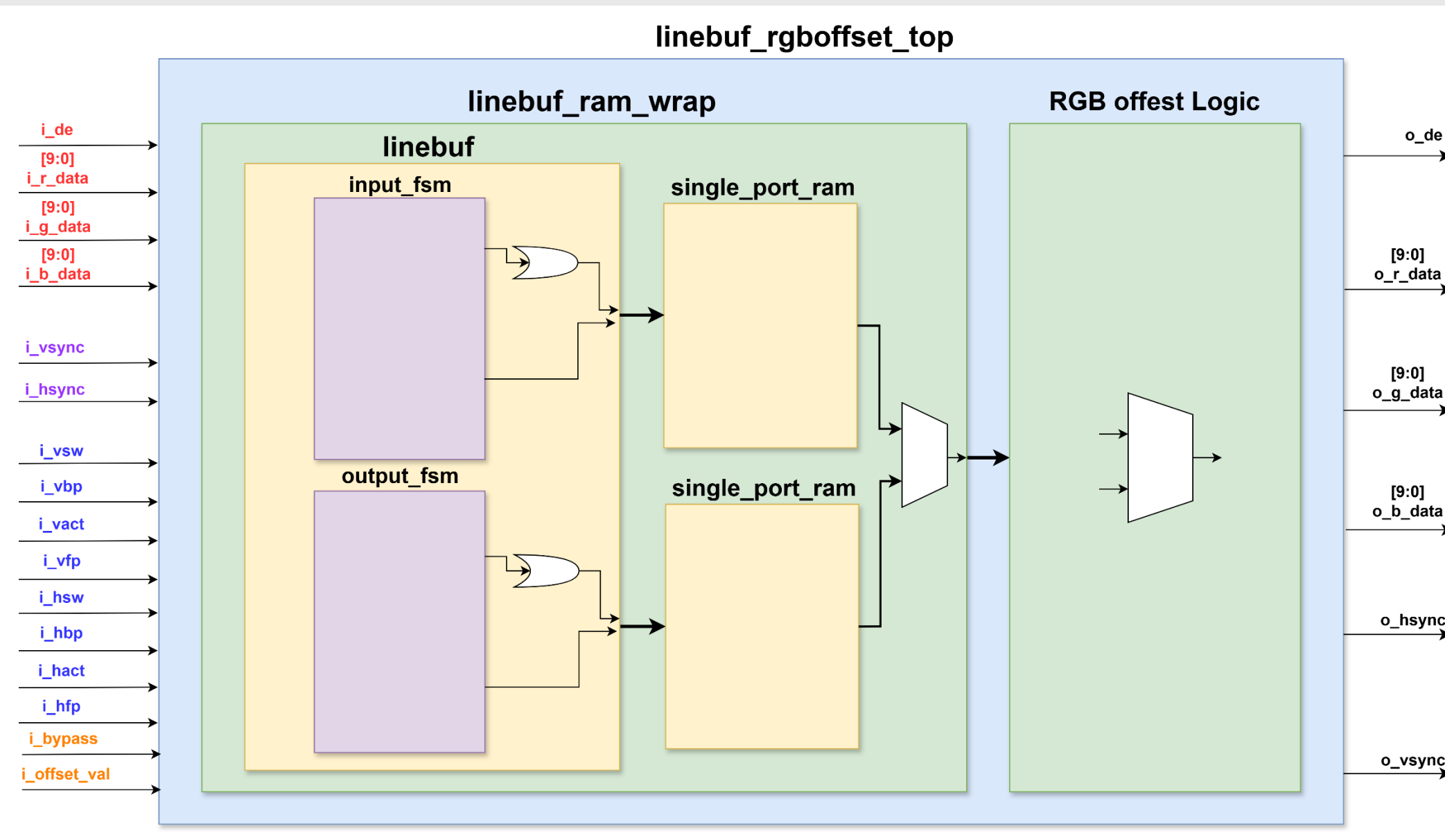




2. DUT Spec Analysis

02

linebuf & linebuf_rgboffset_top module

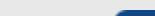


구분	linebuf	linebuf_rgboffset_top
핵심 역할	데이터 지연 및 타이밍 생성	시스템 제어 및 영상 데이터 처리
주요 기능 1	Double Buffering: 2개의 SRAM을 교차 제어하여 실시간 라인 지연(1-Line Latency) 구현	Path Selection (Bypass): 제어 신호 (i_bypass)에 따라 입력 데이터를 즉시 출력할지, 버퍼를 거칠지 결정
주요 기능 2	Timing Re-generation: 설정값 기반으로 표준 비디오 동기 신호(vsync, hsync, de) 재구성	RGB Offset 연산: 각 채널 데이터에 사용자 설정 오프셋(i_offset_val)을 가산하는 영상 처리 수행
주요 기능 3	Memory Arbitration: Input/Output FSM을 통한 SRAM 쓰기/읽기 주소 및 제어권 관리	Data Clamping: 가산 결과가 10-bit 최대값(1023)을 초과할 경우 MAX_VAL로 고정하여 데이터 깨짐 방지
입력 데이터	순수 RGB 픽셀 데이터 및 타이밍 파라미터	Bypass 여부, Offset 설정값을 포함한 전체 제어 신호
출력 데이터	지연된 Raw RGB 데이터 및 재생성된 타이밍	Bypass/Offset이 적용된 최종 가공 데이터 및 파이프라인된 타이밍
데이터 동기	라인 단위 동기화 (Input/Output FSM)	레지스터(r_bypass, r_offset_val)를 통한 제어 신호 안정화



3. Test Planning

03



Test Planning

|—  **Phase 1** : 다양한 Frame Size에서의 정상 동작 확인.

└ VSW, VBP, HSW, HBP, $\frac{H}{W}$ $\frac{H}{W}$

1

|—  **Phase 2** : 다양한 R/G/B Data가 입력되었을 때 정상 동작 확인.

| └─ #1. Random Data: 매 clock마다 R, G, B data에 Random한 값이 입력될 경우

| └─ #2. Fix Data: R, G, B data에 고정된 값이 입력될 경우

| └─ #3. Increasing Data(+Wrapping): 한 frame 동안 R, G, B data가 0부터 증가,
| (단, 최대치를 넘을 경우 다시 0으로 순환 되어야 함.)

1

└─  **Phase 3** : DUT bypass, offset 기능의 정상 동작 확인.



4. TB 작성

04

UVM Sequence

1) base sequence

```
class lb_ro_base_seq_c extends uvm_sequence#(lb_ro_drv_pkt_c);
    `uvm_object_utils(lb_ro_base_seq_c)

    lb_ro_seq_item_c rnd_item;
    virtual lb_ro_if lb_ro_vif;

    rand data_mode_e data_mode;
    rand bit [`RGB_WIDTH-1:0] fix_r_data, fix_g_data, fix_b_data;
    rand bit user_bypass;
    rand bit [`RGB_WIDTH-1:0] user_offset_val;
    rand bit [`VER_WIDTH-1:0] user_vsw, user_vbp, user_vact, user_vfp;
    rand bit [`HOR_WIDTH-1:0] user_hsw, user_hbp, user_hact, user_hfp;

    function new (string name = "lb_ro_base_seq_c");
        super.new(name);
        rnd_item = new();
    endfunction

    task send_signal();
        `uvm_create(req);
        req.i_bypass = rnd_item.i_bypass;
        req.i_offset_val = rnd_item.i_offset_val;
        req.i_vsw = rnd_item.i_vsw;
        req.i_vbp = rnd_item.i_vbp;
        req.i_vact = rnd_item.i_vact;
        req.i_vfp = rnd_item.i_vfp;
        req.i_hsw = rnd_item.i_hsw;
        req.i_hbp = rnd_item.i_hbp;
        req.i_hact = rnd_item.i_hact;
        req.i_hfp = rnd_item.i_hfp;
        req.i_vsync = rnd_item.i_vsync;
        req.i_hsync = rnd_item.i_hsync;
        req.i_de = rnd_item.i_de;
        req.i_r_data = rnd_item.i_r_data;
        req.i_g_data = rnd_item.i_g_data;
        req.i_b_data = rnd_item.i_b_data;
        `uvm_send(req);
    endtask

    task send_init(input int size=1);
        for (int i=1; i<=size; i++) begin
            rnd_item.i_bypass = 0;
            rnd_item.i_offset_val = 0;
            rnd_item.i_vsw = 0;
            rnd_item.i_vbp = 0;
            rnd_item.i_vact = 0;
            rnd_item.i_vfp = 0;
            rnd_item.i_hsw = 0;
            rnd_item.i_hbp = 0;
            rnd_item.i_hact = 0;
            rnd_item.i_hfp = 0;
            rnd_item.i_vsync = 0;
            rnd_item.i_hsync = 0;
            rnd_item.i_de = 0;
            rnd_item.i_r_data = 0;
            rnd_item.i_g_data = 0;
            rnd_item.i_b_data = 0;
            send_signal();
        end
    endtask
endclass
```

- video_test -> vseqr -> seq_item -> seq_lib 과정을 통해서 user가 설정한 paramete 값들을 전달받음.
- 시나리오 실행때마다 frame parameter값들을 전달하기 위해서 rand로 설정

sequence 클래스의 객체를 생성(초기화)함과 동시에, 필요한 데이터(rnd_item)의 메모리 공간을 미리 할당

- send_signal();
Seqeunce에서 생성된 데이터를 req를 통해서 driver로 전달하는 task.

- send_init();
시뮬레이션 시작 시, 모든 입력 신호와 비디오 타이밍 변수들 을 0으로 초기화하는 task.

UVM Sequence

2) user_sequence

p_sequencer 생성 UVM Macro

base_seq에서 받아온 값들을 사용하기 위해 할당

시퀀스 클래스의 객체를 생성(초기화)함

- body();
user_seq의 실행을 위한 task

```
class lb_ro_user_seq_c extends lb_ro_base_seq_c;
  `uvm_object_utils(lb_ro_user_seq_c)
  `uvm_declare_p_sequencer(lb_ro_sequencer_c)
  data_mode_e data_mode;

  bit [`RGB_WIDTH-1:0] fix_r_data, fix_g_data, fix_b_data;
  bit user_bypass;
  bit [`RGB_WIDTH-1:0] user_offset_val;
  bit [`VER_WIDTH-1:0] user_vsw, user_vbp, user_vact, user_vfp;
  bit [`HOR_WIDTH-1:0] user_hsw, user_hbp, user_hact, user_hfp;

  function new(string name = "lb_ro_user_seq_c");
    super.new(name);
  endfunction

  virtual task body();
    if (rnd_item == null) rnd_item = new();

    repeat (4) begin
      vtiming();
    end
    vtiming_end();
    send_signal();
  endtask
endclass
```



UVM Sequence

2) user sequence / vtiming() task

```
virtual task vtiming();
    "uvm_info(get_type_name(), "vtiming starts..", UVM_MEDIUM)
    if (data_mode == INCREASE) begin
        s_r_data = 0;
        s_g_data = 0;
        s_b_data = 0;
    end

    rnd_item.i_vsync = 1;
    rnd_item.i_hsync = 1;
    send_signal();
    repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
    rnd_item.i_hsync = 0;
    send_signal();
    repeat (rnd_item.i_vsw - 1) begin
        repeat (rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp - 1) begin
            @(posedge p_sequencer.lb_ro_vif.i_clk);
        end
        rnd_item.i_hsync = 1;
        send_signal();
        repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
        rnd_item.i_hsync = 0;
        send_signal();
    end
    repeat (rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp - 1) begin
        @(posedge p_sequencer.lb_ro_vif.i_clk);
    end
    repeat (rnd_item.i_vbp) begin
        rnd_item.i_hsync = 1;
        rnd_item.i_de = 0;
        rnd_item.i_vsync = 0;
        repeat (rnd_item.i_hsw) begin
            send_signal();
        end
        rnd_item.i_hsync = 0;
        repeat ((rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp)) begin
            send_signal();
        end
    end
    rnd_item.i_vsync = 0;
    for (int v = 0; v < rnd_item.i_vact; v++) begin
        rnd_item.i_hsync = 1;
        send_signal();
        repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
        rnd_item.i_hsync = 0;
        send_signal();
        repeat (rnd_item.i_hbp - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
        rnd_item.i_de = 1;
        for (int h = 0; h < rnd_item.i_hact; h++) begin
            if (data_mode == FIX) begin
                rnd_item.i_r_data = fix_r_data;
                rnd_item.i_g_data = fix_g_data;
                rnd_item.i_b_data = fix_b_data;
            end else if (data_mode == RANDOM) begin
                {rnd_item.i_r_data, rnd_item.i_g_data, rnd_item.i_b_data} = $random();
            end else if (data_mode == INCREASE) begin
                if (h == 0 & v == 0) begin
                    rnd_item.i_r_data = 0;
                    rnd_item.i_g_data = 0;
                    rnd_item.i_b_data = 0;
                end else begin
                    rnd_item.i_r_data = s_r_data + 5;
                    rnd_item.i_g_data = s_g_data + 5;
                    rnd_item.i_b_data = s_b_data + 5;
                    s_r_data = rnd_item.i_r_data;
                    s_g_data = rnd_item.i_g_data;
                    s_b_data = rnd_item.i_b_data;
                end
            end
            send_signal();
        end
        if (data_mode == INCREASE) begin
            s_r_data = rnd_item.i_r_data;
            s_g_data = rnd_item.i_g_data;
            s_b_data = rnd_item.i_b_data;
        end
        rnd_item.i_de = 0;
        rnd_item.i_r_data = 0;
        rnd_item.i_g_data = 0;
        rnd_item.i_b_data = 0;
        repeat (rnd_item.i_hfp) begin
            send_signal();
        end
    end
end
```

**Data_mode = INCREASE일때,
매 프레임마다 0으로 초기화**

vsw, vbp에 따른 vsync, hsync 데이터 생산을 위한 로직

- vsync 생성 후 vact, vfp, hbp, hfp에 따른 hsync, de 생성하는 video timing Logic
- data_mode에 따라서 생성되는 R/G/B Data는 달라짐



UVM Sequence

2) user sequence / vtiming() task

```
repeat (rnd_item.i_vfp) begin
    rnd_item.i_hsync = 1;
    send_signal();
    repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
    rnd_item.i_hsync = 0;
    send_signal();
    for (
        int v = 0;
        v < (rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp - 1);
        v++
    ) begin
        send_signal();
    end
end
`uvm_info(get_type_name(), "vtiming ends..", UVM_MEDIUM)
endtask
```

- vfp에 따른 hsync 생성

```
virtual task vtiming_end();
    `uvm_info(get_type_name(), "vtiming starts..", UVM_MEDIUM)
    rnd_item.i_vsync = 1;
    rnd_item.i_hsync = 1;
    send_signal();
    repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
    rnd_item.i_hsync = 0;
    send_signal();
    repeat (rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp - 1) begin
        @(posedge p_sequencer.lb_ro_vif.i_clk);
    end
    // forever begin // eda playground에서 동작 x
    repeat(5) begin
        rnd_item.i_hsync = 1;
        send_signal();
        repeat (rnd_item.i_hsw - 1) @(posedge p_sequencer.lb_ro_vif.i_clk);
        rnd_item.i_hsync = 0;
        send_signal();
        repeat (rnd_item.i_hact + rnd_item.i_hfp + rnd_item.i_hbp - 1) begin
            @(posedge p_sequencer.lb_ro_vif.i_clk);
        end
    end
    `uvm_info(get_type_name(), "vtiming ends..", UVM_MEDIUM)
endtask
endclass
```

- 프레임 생성 과정 완료시 대기 위한 로직
(원래는 데이터가 들어오기 전까지 대기해야 하는데
eda_playground에서는 실행 불가)

1) UVM Scoreboard - pixel

```
import "DPI-C" function int unsigned addFunc(int unsigned a, int unsigned b);
```

addFunc.c에 있는 기능을 사용하기 위해 선언

```
uvm_analysis_imp_decl(in_lb_ro)  
uvm_analysis_imp_decl(out_lb_ro)
```

Input Monitor, Output Monitor에서 들어오는 데이터를 구분하여 받기 위한 개별 Port의 이름을 선언

```
class lb_ro_sb_c extends uvm_scoreboard;  
    uvm_component_utils(lb_ro_sb_c)
```

Monitor에서 보낸 데이터를 Scoreboard가 받을 수 있도록 입력과 출력에 대한 TLM Port를 각각 Instance

```
typedef struct packed {  
    bit [9:0] r, g, b;  
    bit      bypass;  
    bit [9:0] offset_val;  
} rgb_set;
```

```
uvm_analysis_imp_in_lb_ro #(lb_ro_mon_pkt_c, lb_ro_sb_c) in_lb_ro_imp_port;  
uvm_analysis_imp_out_lb_ro #(lb_ro_mon_pkt_c, lb_ro_sb_c) out_lb_ro_imp_port;
```

scoreboard의 초기화 단계. 선언된 analysis_imp in/out Port를 new()를 통해 메모리에 할당하고, 이 Port들을 Scoreboard에 소속시키는 과정

```
rgb_set c_result_q[$];  
rgb_set rtl_result_q[$];  
bit ['RGB_WIDTH-1:0] max_value = {'RGB_WIDTH{1'b1}};  
int match_cnt = 0;  
int mismatch_cnt = 0;  
int in_pkt_cnt = 0;  
int out_pkt_cnt = 0;
```

```
function new(string name, uvm_component parent);  
    super.new(name, parent);  
    in_lb_ro_imp_port = new("in_lb_ro_imp_port", this);  
    out_lb_ro_imp_port = new("out_lb_ro_imp_port", this);  
endfunction
```

```
virtual function void write_in_lb_ro(lb_ro_mon_pkt_c pkt);  
    if(pkt.i_de) begin  
        rgb_set expected_data;  
        expected_data.r      = pkt.i_r_data;  
        expected_data.g      = pkt.i_g_data;  
        expected_data.b      = pkt.i_b_data;  
        expected_data.bypass = pkt.i_bypass;  
        expected_data.offset_val = pkt.i_offset_val;  
  
        c_result_q.push_back(expected_data);  
        in_pkt_cnt++;  
    end  
endfunction
```

입력 모니터로부터 받은 데이터 중 i_de=1일때 만 추출하여 예상 결과값(expected_data)으로 가공한 뒤, 추후 비교를 위해 내부 Queue에 순서대로 쌓아두는 함수



UVM Scoreboard

1) UVM Scoreboard - pixel

비교를 위해 저장해 두었던 입력 데이터(Expected)를 꺼내어, 실제 출력값(Actual)과 정답지(Golden Reference)를 만들기 위한 기초 데이터로 설정

- Bypass 모드 여부에 따라 Golden Reference을 생성하는 핵심 검증 로직.
- Bypass = 1 : 입력값을 그대로 정답으로 사용
- Bypass = 0 : 입력값에 Offset을 더한 후 최대치를 넘지 않도록 Clamping 처리

- Scoreboard의 최종 단계
- DUT가 출력한 실제 값(actual_result)과 미리 계산한 정답(golden_ref)을 1:1로 비교하여 검증 통과 여부를 판정하고 그 결과를 display

시뮬레이션이 종료된 후, 검증 결과를 한눈에 볼 수 있도록 최종 결과를 출력하는 리포트 단계(Report Phase)

```
virtual function void write_out_lb_ro(lb_ro_mon_pkt_c pkt);
    rgb_set actual_result;
    rgb_set golden_ref;
    string cmp_msg;

    out_pkt_cnt++;

    actual_result.r = pkt.o_r_data;
    actual_result.g = pkt.o_g_data;
    actual_result.b = pkt.o_b_data;

    if(c_result_q.size() > 0) begin
        rgb_set stored_item = c_result_q.pop_front();
        actual_result.bypass = stored_item.bypass;
        actual_result.offset_val = stored_item.offset_val;

        golden_ref.bypass = stored_item.bypass;
        golden_ref.offset_val = stored_item.offset_val;

        if (stored_item.bypass) begin
            golden_ref.r = stored_item.r;
            golden_ref.g = stored_item.g;
            golden_ref.b = stored_item.b;
            cmp_msg = $sformatf("BYPASS (Orig -> Pure): {R:%h, G:%h, B:%h}", stored_item.r, stored_item.g, stored_item.b);
        end else begin
            int unsigned res_r, res_g, res_b;
            res_r = addFunc(int'(stored_item.r), int'(stored_item.offset_val));
            res_g = addFunc(int'(stored_item.g), int'(stored_item.offset_val));
            res_b = addFunc(int'(stored_item.b), int'(stored_item.offset_val));

            golden_ref.r = (res_r > max_value) ? max_value : res_r[9:0];
            golden_ref.g = (res_g > max_value) ? max_value : res_g[9:0];
            golden_ref.b = (res_b > max_value) ? max_value : res_b[9:0];
            cmp_msg = $sformatf("OFFSET (Orig+Off): {R:%h+%h, G:%h+%h, B:%h+%h}",
                                stored_item.r, stored_item.offset_val,
                                stored_item.g, stored_item.offset_val,
                                stored_item.b, stored_item.offset_val);
        end

        if ((actual_result.r == golden_ref.r) && (actual_result.g == golden_ref.g) && (actual_result.b == golden_ref.b)) begin
            match_cnt++;
            `uvm_info("DATA_CMP", $sformatf("MATCH SUCCESS! [Pkt#%0d] %s -> Expected:%p == Actual:%p",
                                            out_pkt_cnt, cmp_msg, golden_ref, actual_result), UVM_LOW)
        end else begin
            mismatch_cnt++;
            `uvm_error("DATA_CMP", $sformatf("MATCH FAILED! [Pkt#%0d] %s -> Expected:%p != Actual:%p",
                                            out_pkt_cnt, cmp_msg, golden_ref, actual_result))
        end
    end else begin
        `uvm_error("SB_EMPTY", $sformatf("Output [#%0d] received but c_result_q is EMPTY!", out_pkt_cnt))
    end
endfunction

function void report_phase(uvm_phase phase);
    super.report_phase(phase);
    `uvm_info(get_type_name(), "===== ", UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" FINAL REPORT"), UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" - Total Input Pixels : %0d", in_pkt_cnt), UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" - Total Output Pixels : %0d", out_pkt_cnt), UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" - Match Count : %0d", match_cnt), UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" - Mismatch Count : %0d", mismatch_cnt), UVM_LOW)
    `uvm_info(get_type_name(), $sformatf(" - Remaining In Q : %0d", c_result_q.size()), UVM_LOW)
    `uvm_info(get_type_name(), "===== ", UVM_LOW)
endfunction
endclass
```

2) UVM Scoreboard - line

```
import "DPI-C" function int unsigned addFunc(int unsigned a, int unsigned b);

`uvm_analysis_imp_decl(_in_line)
`uvm_analysis_imp_decl(_out_line)

class lb_ro_sb_c extends uvm_scoreboard;
    `uvm_component_utils(lb_ro_sb_c)

    uvm_analysis_imp_in_line #(lb_ro_mon_pkt_c, lb_ro_sb_c) in_line_imp_port;
    uvm_analysis_imp_out_line #(lb_ro_mon_pkt_c, lb_ro_sb_c) out_line_imp_port;

    lb_ro_mon_pkt_c in_line_q[$];
    bit [`RGB_WIDTH-1:0] max_value = {'RGB_WIDTH{1'b1}};
    int match_cnt = 0;
    int mismatch_cnt = 0;
    int line_match_cnt = 0;
    int line_mismatch_cnt = 0;

    function new(string name, uvm_component parent);
        super.new(name, parent);
        in_line_imp_port = new("in_line_imp_port", this);
        out_line_imp_port = new("out_line_imp_port", this);
    endfunction

    virtual function void write_in_line(lb_ro_mon_pkt_c pkt);
        in_line_q.push_back(pkt);
    endfunction
```

line 단위로 저장하기 위한 Queue 생성 및
match/mismatch 여부를 판단하기 위한 변수 선언

line 단위 패킷 데이터를 비교하기 위해 내부
Queue에 순차적으로 저장하는 함수



UVM Scoreboard

2) UVM Scoreboard - line

입력 Queue에서 보관된 데이터를 꺼내 현재 모드와 총 픽셀 수를 포함한 구분선을 출력함으로써, 본격적인 line 단위 데이터 비교가 시작됨을 알리는 요약 로그 출력

동작 모드에 따라 픽셀별 예상값을 계산하고, 원본 데이터와 오프셋 연산 과정 및 실제 출력값을 비교하기 위한 상세 로그 문자열을 생성하는 루프 로직

개별 픽셀의 비교 결과를 상세 로그로 출력하고 일치 여부에 따라 pixel 및 line 단위 카운터를 갱신

입력 데이터 부재 시 에러를 발생시키는 검증 판단 로직

시뮬레이션 종료 단계에서 pixel, line 단위로 누적된 전체 검증 성공 및 실패 횟수를 요약하여 최종 리포트로 출력하는 함수

```
virtual function void write_out_line(lb_mon_pkt_c pkt);
if(in_line_q.size() > 0) begin
    lb_mon_pkt_c exp = in_line_q.pop_front();
    bit ok = 1;
    string mode_str = exp.i_bypass ? "BYPASS" : $sprintf("OFFSET(Val: %0d)", exp.i_offset_val);
    int pxl_cnt = pkt.r_payload.size();

    `uvm_info("SB_LINE", "===== ", UVM_LOW)
    `uvm_info("SB_LINE", $sprintf(" Time: %0t | Mode: %s | Total Pixels: %0d", $time, mode_str, pxl_cnt), UVM_LOW)
    `uvm_info("SB_LINE", "===== ", UVM_LOW)

    foreach(pkt.r_payload[i]) begin
        bit [9:0] er, eg, eb;
        string msg, r_str, g_str, b_str;

        if(exp.i_bypass) begin
            er = exp.r_payload[i]; eg = exp.g_payload[i]; eb = exp.b_payload[i];

            r_str = $sprintf("R:[Exp:%h]Act:%h", er, pkt.r_payload[i]);
            g_str = $sprintf("G:[Exp:%h]Act:%h", eg, pkt.g_payload[i]);
            b_str = $sprintf("B:[Exp:%h]Act:%h", eb, pkt.b_payload[i]);
        end else begin
            er = (addFunc(int'(exp.r_payload[i]), int'(exp.i_offset_val)) > max_value) ? max_value : (exp.r_payload[i] + exp.i_offset_val);
            eg = (addFunc(int'(exp.g_payload[i]), int'(exp.i_offset_val)) > max_value) ? max_value : (exp.g_payload[i] + exp.i_offset_val);
            eb = (addFunc(int'(exp.b_payload[i]), int'(exp.i_offset_val)) > max_value) ? max_value : (exp.b_payload[i] + exp.i_offset_val);

            r_str = $sprintf("R:[%h + %0d = %h | Act: %h]", exp.r_payload[i], exp.i_offset_val, er, pkt.r_payload[i]);
            g_str = $sprintf("G:[%h + %0d = %h | Act: %h]", exp.g_payload[i], exp.i_offset_val, eg, pkt.g_payload[i]);
            b_str = $sprintf("B:[%h + %0d = %h | Act: %h]", exp.b_payload[i], exp.i_offset_val, eb, pkt.b_payload[i]);
        end

        msg = $sprintf("[Px1 %3d] %s %s %s", i, r_str, g_str, b_str);

        if(pkt.r_payload[i] == er && pkt.g_payload[i] == eg && pkt.b_payload[i] == eb) begin
            match_cnt++;
            `uvm_info("SB_MATCH", msg, UVM_LOW)
        end else begin
            ok = 0;
            mismatch_cnt++;
            `uvm_error("SB_MISMATCH", msg)
        end
    end

    if(ok) line_match_cnt++;
    else line_mismatch_cnt++;

    `uvm_info("SB_LINE", "===== \n", UVM_LOW)
end else begin
    `uvm_error("SB_NO_INPUT", "Input data not found in Scoreboard Queue!")
end
endfunction

function void report_phase(uvm_phase phase);
super.report_phase(phase);
`uvm_info(get_type_name(), "-----", UVM_LOW)
`uvm_info(get_type_name(), $sprintf(" Pixel Match/Mismatch : %0d / %0d", match_cnt, mismatch_cnt), UVM_LOW)
`uvm_info(get_type_name(), $sprintf(" Line Match/Mismatch : %0d / %0d", line_match_cnt, line_mismatch_cnt), UVM_LOW)
`uvm_info(get_type_name(), "-----", UVM_LOW)
endfunction
endclass
```

3) UVM Scoreboard - frame

```
import "DPI-C" function int unsigned addFunc(int unsigned a, int unsigned b);

`uvm_analysis_imp_decl(_in_line)
`uvm_analysis_imp_decl(_out_line)

class lb_ro_sb_c extends uvm_scoreboard;
  `uvm_component_utils(lb_ro_sb_c)

  uvm_analysis_imp_in_line #(lb_ro_mon_pkt_c, lb_ro_sb_c) in_line_imp_port;
  uvm_analysis_imp_out_line #(lb_ro_mon_pkt_c, lb_ro_sb_c) out_line_imp_port;

  typedef lb_ro_mon_pkt_c frame_t[$];
  lb_ro_mon_pkt_c in_line_q[$];
  frame_t in_frame_q[$];

  bit [`RGB_WIDTH-1:0] max_value = {'RGB_WIDTH{1'b1}};

  int match_cnt = 0;
  int mismatch_cnt = 0;
  int line_match_cnt = 0;
  int line_mismatch_cnt = 0;
  int frame_match_cnt = 0;
  int frame_mismatch_cnt = 0;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    in_line_imp_port = new("in_line_imp_port", this);
    out_line_imp_port = new("out_line_imp_port", this);
  endfunction

  virtual function void write_in_line(lb_ro_mon_pkt_c pkt);
    int local_vact;
    local_vact = pkt.i_vact;
    in_line_q.push_back(pkt);
    if (in_line_q.size() == local_vact) begin
      in_frame_q.push_back(in_line_q);
      in_line_q.delete();
    end
  endfunction

  virtual function void write_out_line(lb_ro_mon_pkt_c pkt);
    static frame_t current_out_frame;
    int local_vact;
    local_vact = | pkt.i_vact;
    current_out_frame.push_back(pkt);
    if (current_out_frame.size() == local_vact) begin
      compare_frame(current_out_frame);
      current_out_frame.delete();
    end
  endfunction
```

수신된 line 패킷을 내부 Queue에 저장한 후,
local_vact에 도달하면 해당 line 그룹을 하나의 frame
단위로 묶어 별도의 frame Queue로 관리하는 로직

출력 모니터로부터 수신한 line 패킷을 정적 변수인
current_out_frame에 누적한 후,
i_vact에 도달하면 frame 단위 비교 함수를 호출하
고 Queue를 비우는 기능



UVM Scoreboard

3) UVM Scoreboard - frame

```
function void compare_frame(frame_t exp_frame)
frame_t exp_frame;
lb_ro_mon_pkt_c exp_line;
lb_ro_mon_pkt_c act_line;
bit [RGB_WIDTH-1:0] er, eg, eb;
bit [RGB_WIDTH-1:0] ar, ag, ab;
bit frame_ok;
bit line_ok;

frame_ok = 1;

if (in_frame_q.size() > 0) begin
exp_frame = in_frame_q.pop_front();

foreach (out_frame[l_idx]) begin
exp_line = exp_frame[l_idx];
act_line = out_frame[l_idx];
line_ok = 1;

foreach (act_line.r_payload[p_idx]) begin
ar = act_line.r_payload[p_idx];
ag = act_line.g_payload[p_idx];
ab = act_line.b_payload[p_idx];

if (exp_line.i_bypass) begin
er = exp_line.r_payload[p_idx];
eg = exp_line.g_payload[p_idx];
eb = exp_line.b_payload[p_idx];

`uvm_info("DATA_VAL", $sformatf("L:%0d P:%0d [BYPASS] | EXP: {er, eg, eb} | ACT: {ar, ag, ab}",
l_idx, p_idx, er, eg, eb, ar, ag, ab), UVM_LOW)
end else begin
er = (addFunc(int'(exp_line.r_payload[p_idx]), int'(exp_line.i_offset_val)) > max_value) ? max_value : (exp_line.r_payload[p_idx] + exp_line.i_offset_val);
eg = (addFunc(int'(exp_line.g_payload[p_idx]), int'(exp_line.i_offset_val)) > max_value) ? max_value : (exp_line.g_payload[p_idx] + exp_line.i_offset_val);
eb = (addFunc(int'(exp_line.b_payload[p_idx]), int'(exp_line.i_offset_val)) > max_value) ? max_value : (exp_line.b_payload[p_idx] + exp_line.i_offset_val);

`uvm_info("DATA_VAL", $sformatf("L:%0d P:%0d [OFFSET:%0d] | ORG: {er, eg, eb} -> EXP: {ar, ag, ab} | ACT: {er, eg, eb}",
l_idx, p_idx, exp_line.i_offset_val,
exp_line.r_payload[p_idx], exp_line.g_payload[p_idx], exp_line.b_payload[p_idx],
er, eg, eb, ar, ag, ab), UVM_LOW)
end

if (ar == er && ag == eg && ab == eb) begin
match_cnt++;
end else begin
line_ok = 0;
mismatch_cnt++;
`uvm_error("MISMATCH", $sformatf("Data Error at L:%0d P:%0d", l_idx, p_idx))
end
end

if (line_ok) line_match_cnt++;
else begin
line_mismatch_cnt++;
frame_ok = 0;
end
end

if (frame_ok) frame_match_cnt++;
else frame_mismatch_cnt++;

end else begin
`uvm_error("SB_EMPTY", "Golden frame queue is empty!")
end
endfunction

virtual function void report_phase(uvm_phase phase);
super.report_phase(phase);
`uvm_info(get_type_name(), "===== ", UVM_LOW)
`uvm_info(get_type_name(), $sformatf(" Total Pixel Match/Mismatch : %0d / %0d", match_cnt, mismatch_cnt), UVM_LOW)
`uvm_info(get_type_name(), $sformatf(" Total Line Match/Mismatch : %0d / %0d", line_match_cnt, line_mismatch_cnt), UVM_LOW)
`uvm_info(get_type_name(), $sformatf(" Total Frame Match/Mismatch : %0d / %0d", frame_match_cnt, frame_mismatch_cnt), UVM_LOW)
`uvm_info(get_type_name(), "===== ", UVM_LOW)
endfunction
endclass
```

frame 및 line 단위 비교 과정에서 사용할 예상 데이터와 실제 데이터, 그리고 각 단계의 검증 성공 여부 (match/mismatch)를 저장하기 위한 변수 선언

frame 내의 각 line을 순회하며 예상 패킷과 실제 패킷을 매칭하고, 다시 line 내부의 개별 픽셀 데이터를 추출하여 비교를 준비하는 이중 루프 처리 로직

Bypass 모드일 때는 원본을, Offset 모드일 때는 Saturation 이 적용된 연산값을 예상값으로 설정하여 실제 출력값과 비교하고 결과에 따라 카운터를 갱신하며 로그를 남기는 로직

line 단위의 비교 결과를 바탕으로 line 카운터를 갱신하고, 단 하나의 line이라도 실패할 경우 해당 frame을 실패로 처리하여 최종 frame 통계를 기록하는 검증 계층 구조의 마무리

시뮬레이션 종료 단계에서 pixel, line, frame 단위로 누적된 전체 검증 성공 및 실패 횟수를 요약하여 최종 리포트로 출력하는 함수



5.
시나리오별
시뮬레이션 결과

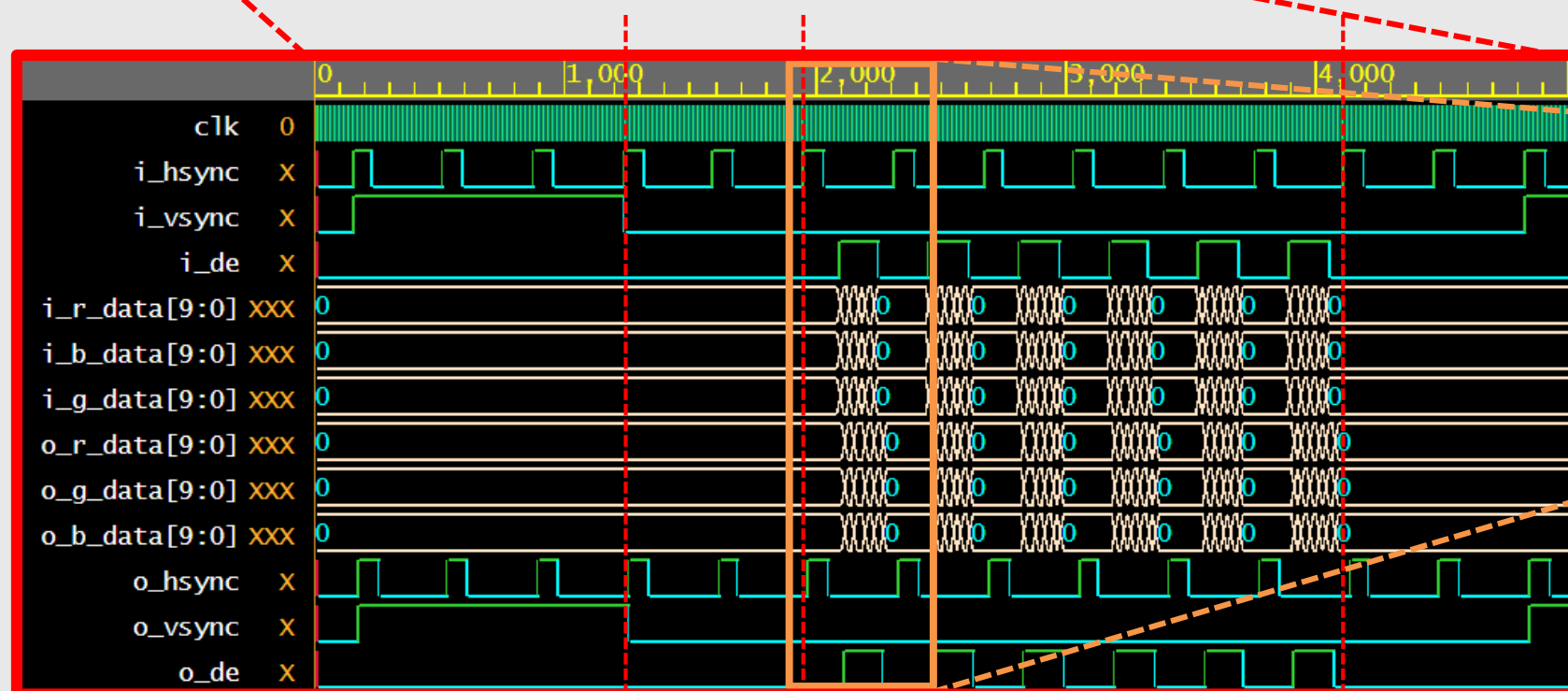
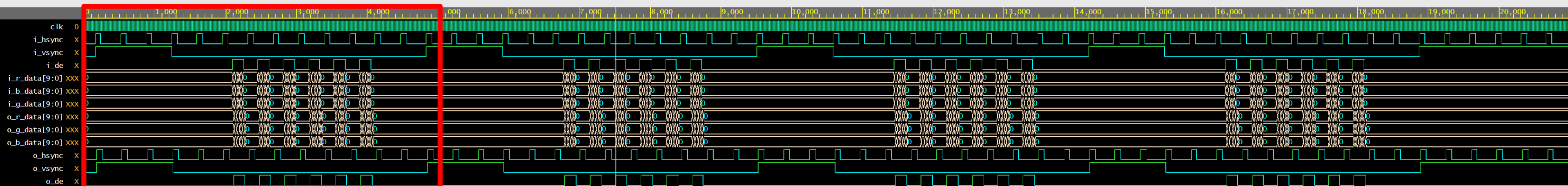
05

시나리오별 시뮬레이션 결과

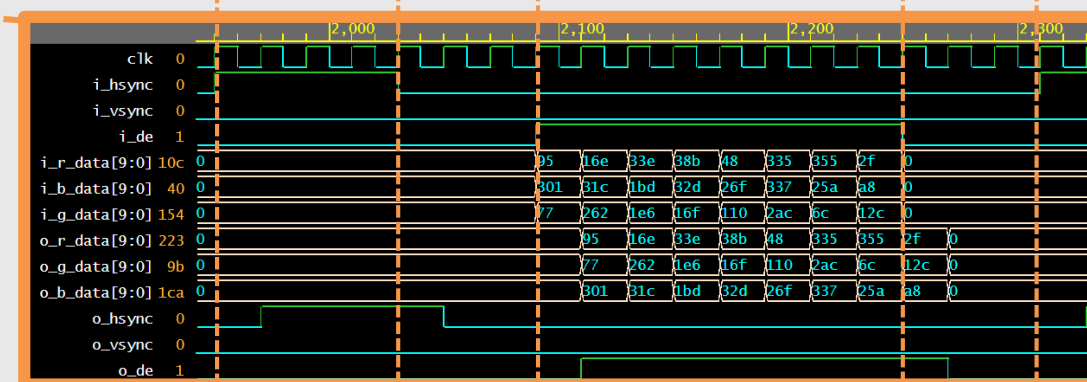
검증 요구사항 #1 확인 (다양한 Frame Size)

Test Case 1: (VSW = 3, VBP = 2, VACT = 6, VFP = 2
HSW = 4, HBP = 3, HACT = 8, HFP = 3)

data_mode = RANDOM
user_bypass = 1



VSW (3)
VBP (2)
VACT (6)
VFP (2)



HSW (4)
HBP (3)
HACT (8)
HFP (3)

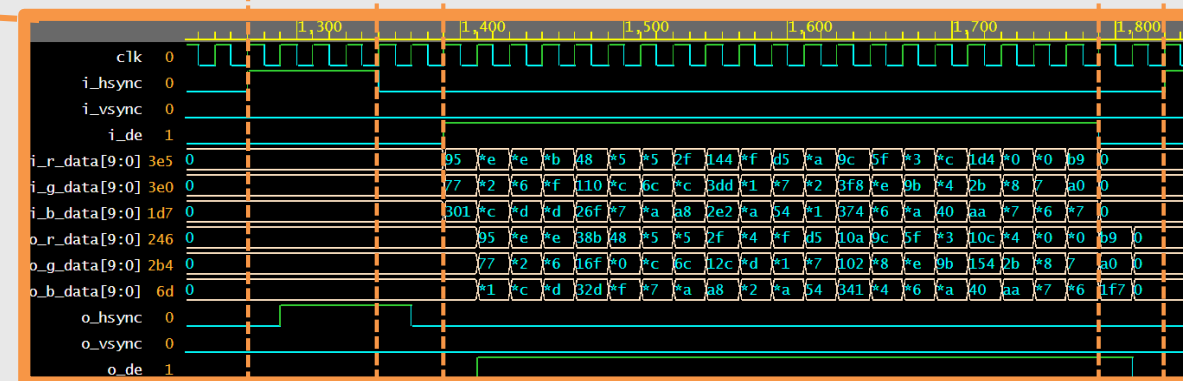
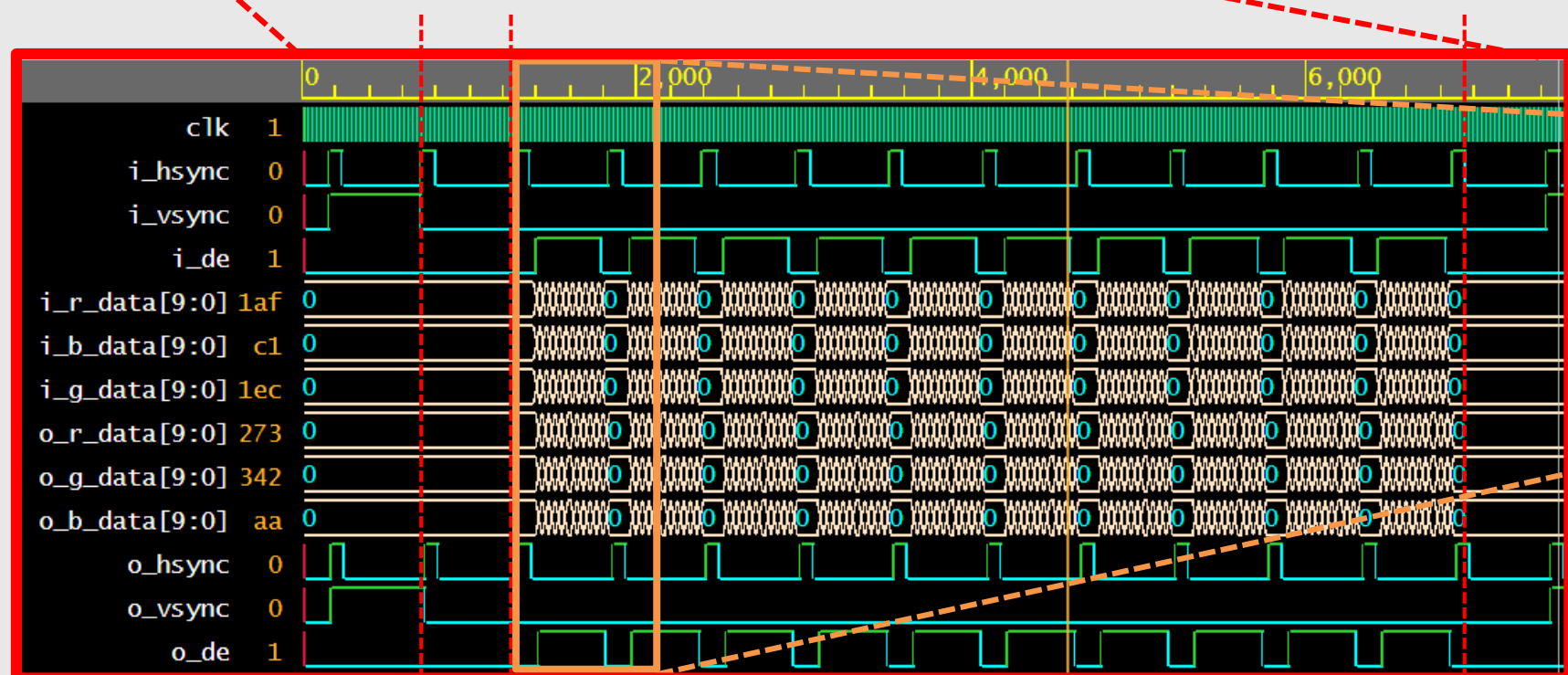
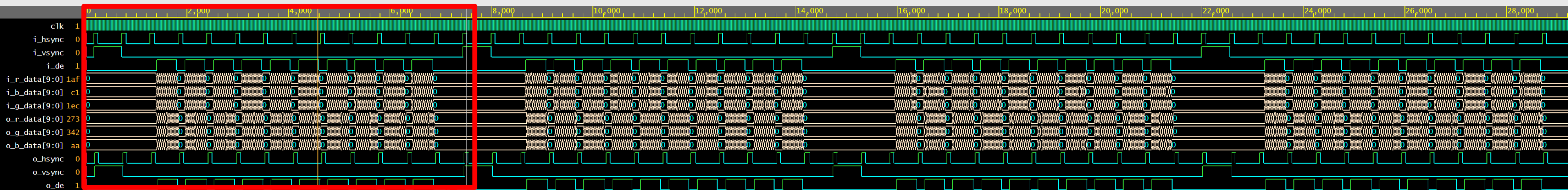
```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,  
{  
    data_mode == RANDOM ;//I  
    fix_r_data == 10 ;  
    fix_g_data == 10 ;  
    fix_b_data == 10 ;  
    user_bypass == 1 ;  
    user_offset_val == 0 ;  
    user_vsw == 3 ;  
    user_vbp == 2 ;  
    user_vact == 6 ;  
    user_vfp == 2 ;  
    user_hsw == 4 ;  
    user_hbp == 3 ;  
    user_hact == 8 ;  
    user_hfp == 3 ;  
})
```


시나리오별 시뮬레이션 결과

검증 요구사항 #1 확인 (다양한 Frame Size)

Test Case 2: (VSW = 1, VBP = 1, VACT = 10, VFP = 1
HSW = 4, HBP = 2, HACT = 20, HFP = 2)

data_mode = RANDOM
user_bypass = 1



```
uvvm_do_on_with(1b_ro_vseq, p_sequencer.1b_ro_seqr,
{
    data_mode      == RANDOM ;//I
    fix_r_data     == 10 ;
    fix_g_data     == 10 ;
    fix_b_data     == 10 ;
    user_bypass    == 1 ;
    user_offset_val == 0 ;
    user_vsw       == 1 ;
    user_vbp       == 1 ;
    user_vact      == 10 ;
    user_vfp       == 1 ;
    user_hsw       == 4 ;
    user_hbp       == 2 ;
    user_hact      == 20 ;
    user_hfp       == 2 ;
})
```

VSW (1) VBP (1) VACT (10) VFP (1)

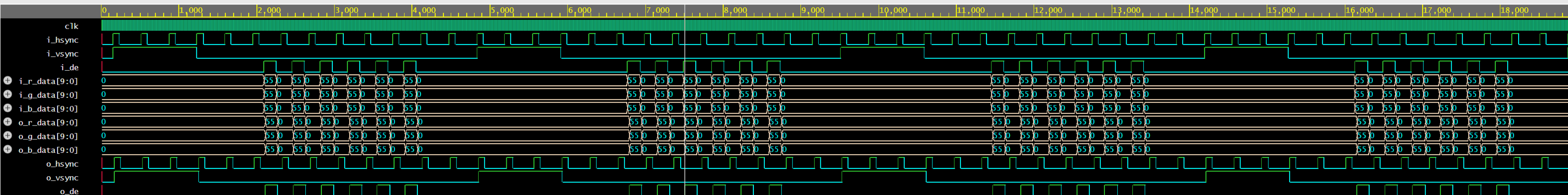
HSW (4) HBP (2) HACT (20) HFP (2)

시나리오별 시뮬레이션 결과

검증 요구사항 #2 확인 (다양한 RGB Data)

Test Case 1: data_mode = FIX (fix_data = 0x55)

(user_bypass = 1,
VSW = 3, VBP = 2, VACT = 6, VFP = 2
HSW = 4, HBP = 3, HACT = 8, HFP = 3)



```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == FIX ; //INC
    fix_r_data     == 10'h55
    fix_g_data     == 10'h55
    fix_b_data     == 10'h55
    user_bypass    == 1 ; /
    user_offset_val == 0 ;
    user_vsw       == 1 ;
    user_vbp       == 1 ;
    user_vact      == 4 ;
    user_vfp       == 1 ;
    user_hsw       == 2 ;
    user_hbp       == 2 ;
    user_hact      == 12 ;
    user_hfp       == 2 ;
})
```

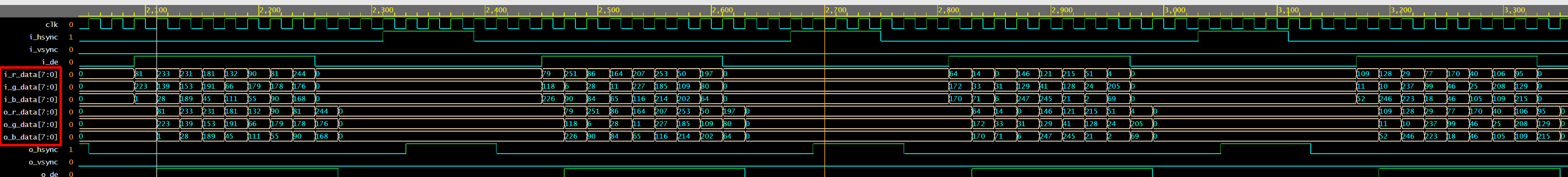
시나리오별 시뮬레이션 결과

검증 요구사항 #2 확인 (다양한 RGB Data)

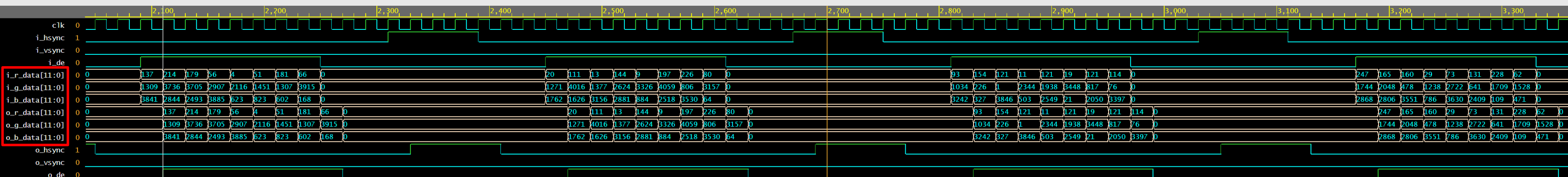
Test Case 2: data_mode = RANDOM

(user_bypass = 1,
VSW = 3, VBP = 2, VACT = 6, VFP = 2
HSW = 4, HBP = 3, HACT = 8, HFP = 3)

1) RGB_WIDTH = 8 (Modified lb_ro_define.sv)



2) RGB_WIDTH = 12 (Modified lb_ro_define.sv)



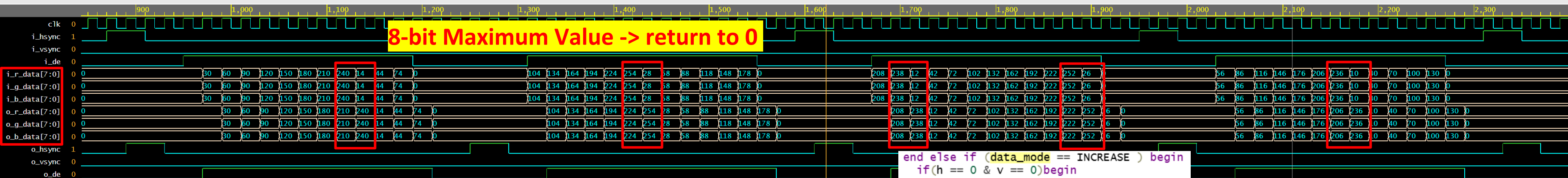
시나리오별 시뮬레이션 결과

검증 요구사항 #2 확인 (다양한 RGB Data)

Test Case 3: data_mode = INCREASE

1) INC_VAL = 30, RGB_WIDTH = 8

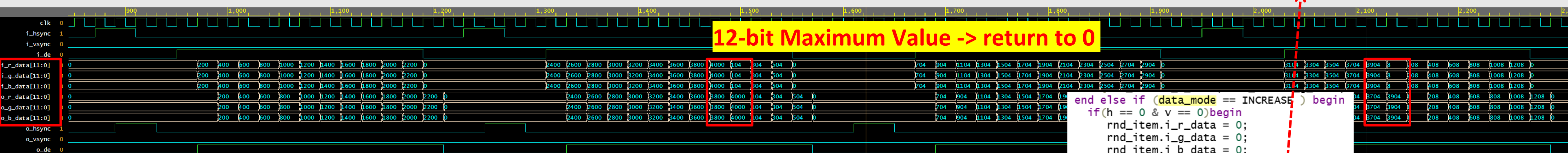
(user_bypass = 1,
VSW = 1, VBP = 1, VACT = 4, VFP = 1
HSW = 2, HBP = 2, HACT = 12, HFP = 2)



```
end else if (data_mode == INCREASE) begin
  if(h == 0 & v == 0)begin
    rnd_item.i_r_data = 0;
    rnd_item.i_g_data = 0;
    rnd_item.i_b_data = 0;
  end else begin
    rnd_item.i_r_data = s_r_data + 30;
    rnd_item.i_g_data = s_g_data + 30;
    rnd_item.i_b_data = s_g_data + 30;
    s_r_data = rnd_item.i_r_data;
    s_g_data = rnd_item.i_g_data;
    s_b_data = rnd_item.i_b_data;
  end
end
```

vtiming() in
lb_ro_seq_lib.sv

2) INC_VAL = 200, RGB_WIDTH = 12



```
end else if (data_mode == INCREASE) begin
  if(h == 0 & v == 0)begin
    rnd_item.i_r_data = 0;
    rnd_item.i_g_data = 0;
    rnd_item.i_b_data = 0;
  end else begin
    rnd_item.i_r_data = s_r_data + 200;
    rnd_item.i_g_data = s_g_data + 200;
    rnd_item.i_b_data = s_g_data + 200;
    s_r_data = rnd_item.i_r_data;
    s_g_data = rnd_item.i_g_data;
    s_b_data = rnd_item.i_b_data;
  end
end
send_signal();
```

시나리오별 시뮬레이션 결과

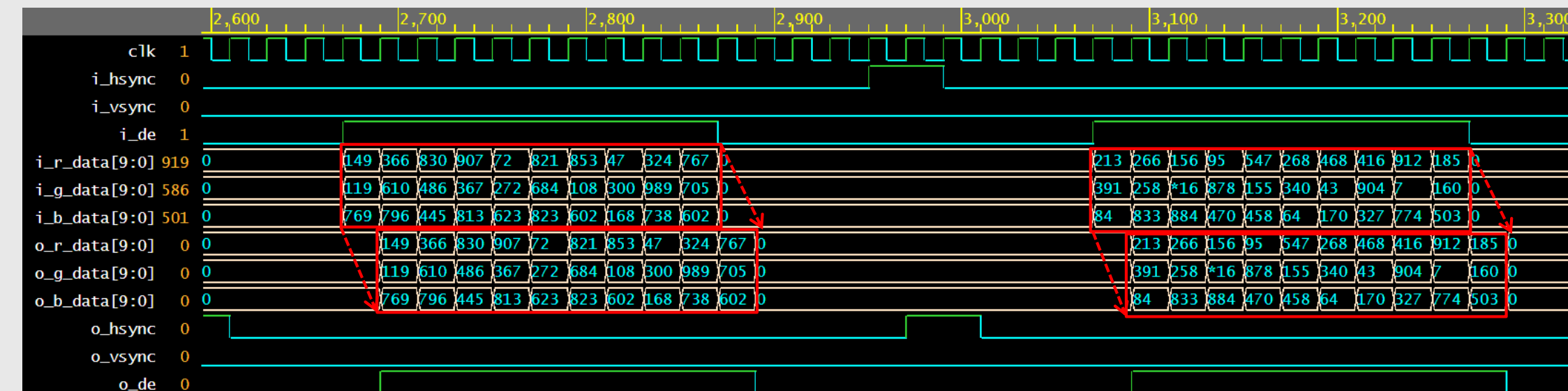
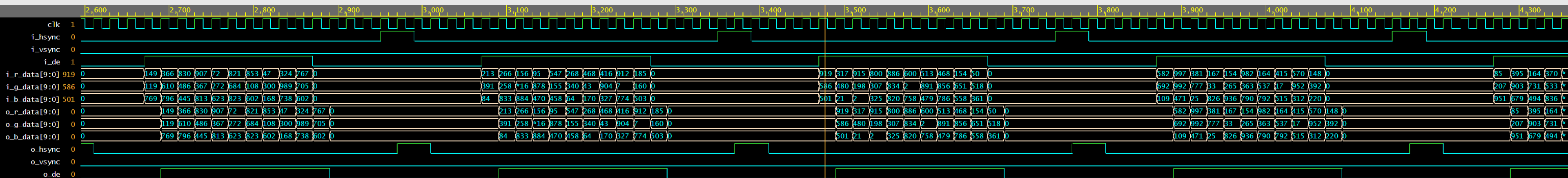
검증 요구사항 #3 확인 (Bypass / Offset mode)

Test Case 1: user_bypass = 1 (Bypass mode)

(data_mode = RANDOM

VSW = 4, VBP = 2, VACT = 8, VFP = 2

HSW = 2, HBP = 4, HACT = 10, HFP = 4)



```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM    ; // INCREASE ;
    fix_r_data     == 10        ;
    fix_g_data     == 10        ;
    fix_b_data     == 10        ;
    user_bypass    == 1         ; // Bypass
    user_offset_val == 0         ;
    user_vsw       == 4         ;
    user_vbp       == 2         ;
    user_vact      == 8         ;
    user_vfp       == 2         ;
    user_hsw       == 2         ;
    user_hbp       == 4         ;
    user_hact      == 10        ;
    user_hfp       == 4         ;
})
```

시나리오별 시뮬레이션 결과

검증 요구사항 #3 확인 (Bypass / Offset mode)

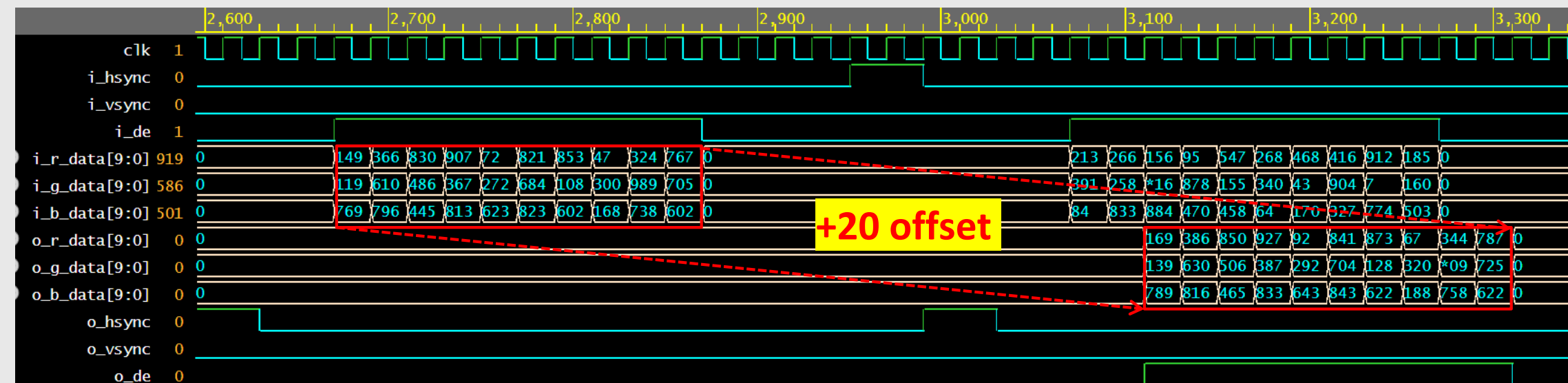
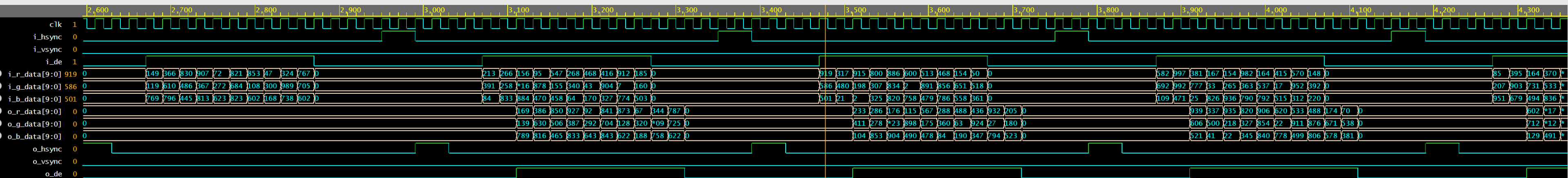
Test Case 2: user_bypass = 0 (offset mode)

user_offset_val = 20

(data_mode = RANDOM

VSW = 4, VBP = 2, VACT = 8, VFP = 2

HSW = 2, HBP = 4, HACT = 10, HFP = 4)



```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM    ; // INCREASE ;
    fix_r_data     == 10        ;
    fix_g_data     == 10        ;
    fix_b_data     == 10        ;
    user_bypass    == 0         ; // offset
    user_offset_val == 20        ;
    user_vsw       == 4         ;
    user_vbp       == 2         ;
    user_vact      == 8         ;
    user_vfp       == 2         ;
    user_hsw       == 2         ;
    user_hbp       == 4         ;
    user_hact      == 10        ;
    user_hfp       == 4         ;
})
```



6. 시나리오별 **Report** 결과

06

1) Report - Compare Conduction

※ Compare C Ref Model 확인

```
#include <stdio.h>

int unsigned addFunc(int unsigned a, int unsigned b) {
    return (a + b);
}
```

구분	비교 방식	주요 검증 포인트
Pixel Compare	Pixel + Offset = Real Pixel Value	산술 연산 정확도
Line Compare	(Pixel + Offset) * Line = Total Pixel Sum Value	픽셀들의 집합 일치성, 가로 해상도
Frame Compare	Each (Pixel + Offset) * Line = Each Line Value	라인 누락 여부, 세로 해상도

시나리오별 Report 결과

2) Report - Scenario1 - 다양한 frame size에서 검증

```
UVM_INFO lb_ro_seq.lib.sv(91) @ 19330: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 21910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#241] BYPASS (Orig -> Pure): {R:299, G:126, B:2ec} -> Expected: {r:'h299, g:'h126, b:'h2ec, bypass:'h1, offset_val:'h14} == Actual: {r:'h299, g:'h126, b:'h2ec, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21930: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#242] BYPASS (Orig -> Pure): {R:323, G:324, B:245} -> Expected: {r:'h323, g:'h324, b:'h245, bypass:'h1, offset_val:'h14} == Actual: {r:'h323, g:'h324, b:'h245, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21950: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#243] BYPASS (Orig -> Pure): {R:318, G:3f9, B:3d2} -> Expected: {r:'h318, g:'h3f9, b:'h3d2, bypass:'h1, offset_val:'h14} == Actual: {r:'h318, g:'h3f9, b:'h3d2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21970: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#244] BYPASS (Orig -> Pure): {R:347, G:00f, B:136} -> Expected: {r:'h347, g:'hf, b:'h136, bypass:'h1, offset_val:'h14} == Actual: {r:'h347, g:'hf, b:'h136, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21990: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#245] BYPASS (Orig -> Pure): {R:24f, G:362, B:216} -> Expected: {r:'h24f, g:'h362, b:'h216, bypass:'h1, offset_val:'h14} == Actual: {r:'h24f, g:'h362, b:'h216, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22010: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#246] BYPASS (Orig -> Pure): {R:0e6, G:19c, B:1a2} -> Expected: {r:'he6, g:'h19c, b:'h1a2, bypass:'h1, offset_val:'h14} == Actual: {r:'he6, g:'h19c, b:'h1a2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22030: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#247] BYPASS (Orig -> Pure): {R:0a8, G:1fe, B:116} -> Expected: {r:'ha8, g:'h1fe, b:'h116, bypass:'h1, offset_val:'h14} == Actual: {r:'ha8, g:'h1fe, b:'h116, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22050: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#248] BYPASS (Orig -> Pure): {R:290, G:2f1, B:320} -> Expected: {r:'h290, g:'h2f1, b:'h320, bypass:'h1, offset_val:'h14} == Actual: {r:'h290, g:'h2f1, b:'h320, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22070: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#249] BYPASS (Orig -> Pure): {R:29d, G:068, B:2ef} -> Expected: {r:'h29d, g:'h68, b:'h2ef, bypass:'h1, offset_val:'h14} == Actual: {r:'h29d, g:'h68, b:'h2ef, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#250] BYPASS (Orig -> Pure): {R:33c, G:251, B:256} -> Expected: {r:'h33c, g:'h251, b:'h256, bypass:'h1, offset_val:'h14} == Actual: {r:'h33c, g:'h251, b:'h256, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22310: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#251] BYPASS (Orig -> Pure): {R:377, G:1bc, B:149} -> Expected: {r:'h377, g:'h1bc, b:'h149, bypass:'h1, offset_val:'h14} == Actual: {r:'h377, g:'h1bc, b:'h149, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#252] BYPASS (Orig -> Pure): {R:17e, G:20d, B:1a0} -> Expected: {r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14} == Actual: {r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22350: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#253] BYPASS (Orig -> Pure): {R:36c, G:016, B:0e2} -> Expected: {r:'h36c, g:'h16, b:'he2, bypass:'h1, offset_val:'h14} == Actual: {r:'h36c, g:'h16, b:'he2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22370: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#254] BYPASS (Orig -> Pure): {R:07a, G:147, B:282} -> Expected: {r:'h7a, g:'h147, b:'h282, bypass:'h1, offset_val:'h14} == Actual: {r:'h7a, g:'h147, b:'h282, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22390: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#255] BYPASS (Orig -> Pure): {R:0d2, G:2bf, B:3da} -> Expected: {r:'hd2, g:'h2bf, b:'h3da, bypass:'h1, offset_val:'h14} == Actual: {r:'hd2, g:'h2bf, b:'h3da, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22410: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#256] BYPASS (Orig -> Pure): {R:051, G:15d, B:3cc} -> Expected: {r:'h51, g:'h15d, b:'h3cc, bypass:'h1, offset_val:'h14} == Actual: {r:'h51, g:'h15d, b:'h3cc, bypass:'h1, offset_val:'h14}
```

```
UVM_INFO lb_ro_sb.sv(101) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(103) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(104) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(105) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(106) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(107) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(108) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

Total Pixel 수
= hact*vact*frame 수
= 10 * 8 * 4 = 320

```
FINAL REPORT
- Total Input Pixels : 320
- Total Output Pixels : 320
- Match Count : 320
- Mismatch Count : 0
- Remaining In Q : 0
```

```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ; //INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10 ;
    fix_g_data     == 10 ;
    fix_b_data     == 10 ;
    user_bypass    == 1 ;
    user_offset_val == 20 ;
    user_vsw       == 4 ;
    user_vbp       == 2 ;
    user_vact      == 8 ;
    user_vfp       == 2 ;
    user_hsw       == 2 ;
    user_hbp       == 4 ;
    user_hact      == 10 ;
    user_hfp       == 4 ;
})
```


시나리오별 Report 결과

2) Report - Scenario1 - 다양한 frame size에서 검증

```
UVM_INFO lb_ro_seq_lib.sv(91) @ 13090: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 15430: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#97] BYPASS (Orig -> Pure): {R:196, G:1a5, B:06f} -> Expected:{r:'h196, g:'h1a5, b:'h6f, bypass:'h1, offset_val:'h14} == Actual:{r:'h196, g:'h1a5, b:'h6f, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15450: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#98] BYPASS (Orig -> Pure): {R:13c, G:144, B:0c4} -> Expected:{r:'h13c, g:'h144, b:'hc4, bypass:'h1, offset_val:'h14} == Actual:{r:'h13c, g:'h144, b:'hc4, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15470: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#99] BYPASS (Orig -> Pure): {R:0a0, G:1a6, B:287} -> Expected:{r:'ha0, g:'h1a6, b:'h287, bypass:'h1, offset_val:'h14} == Actual:{r:'ha0, g:'h1a6, b:'h287, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15490: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#100] BYPASS (Orig -> Pure): {R:23a, G:053, B:0b9} -> Expected:{r:'h23a, g:'h53, b:'hb9, bypass:'h1, offset_val:'h14} == Actual:{r:'h23a, g:'h53, b:'hb9, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15510: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#101] BYPASS (Orig -> Pure): {R:046, G:380, B:096} -> Expected:{r:'h46, g:'h380, b:'h96, bypass:'h1, offset_val:'h14} == Actual:{r:'h46, g:'h380, b:'h96, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15530: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#102] BYPASS (Orig -> Pure): {R:19c, G:2aa, B:031} -> Expected:{r:'h19c, g:'h2aa, b:'h31, bypass:'h1, offset_val:'h14} == Actual:{r:'h19c, g:'h2aa, b:'h31, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15550: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#103] BYPASS (Orig -> Pure): {R:25f, G:2cd, B:082} -> Expected:{r:'h25f, g:'h2cd, b:'h82, bypass:'h1, offset_val:'h14} == Actual:{r:'h25f, g:'h2cd, b:'h82, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15570: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#104] BYPASS (Orig -> Pure): {R:0da, G:3b4, B:2de} -> Expected:{r:'hda, g:'h3b4, b:'h2de, bypass:'h1, offset_val:'h14} == Actual:{r:'hda, g:'h3b4, b:'h2de, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15790: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#105] BYPASS (Orig -> Pure): {R:152, G:09f, B:093} -> Expected:{r:'h152, g:'h9f, b:'h93, bypass:'h1, offset_val:'h14} == Actual:{r:'h152, g:'h9f, b:'h93, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15810: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#106] BYPASS (Orig -> Pure): {R:2a5, G:2f4, B:01f} -> Expected:{r:'h2a5, g:'h2f4, b:'h1f, bypass:'h1, offset_val:'h14} == Actual:{r:'h2a5, g:'h2f4, b:'h1f, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#107] BYPASS (Orig -> Pure): {R:3f7, G:009, B:23b} -> Expected:{r:'h3f7, g:'h9, b:'h23b, bypass:'h1, offset_val:'h14} == Actual:{r:'h3f7, g:'h9, b:'h23b, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15850: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#108] BYPASS (Orig -> Pure): {R:03d, G:25f, B:1ac} -> Expected:{r:'h3d, g:'h25f, b:'h1ac, bypass:'h1, offset_val:'h14} == Actual:{r:'h3d, g:'h25f, b:'h1ac, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15870: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#109] BYPASS (Orig -> Pure): {R:2d4, G:097, B:3a0} -> Expected:{r:'h2d4, g:'h97, b:'h3a0, bypass:'h1, offset_val:'h14} == Actual:{r:'h2d4, g:'h97, b:'h3a0, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15890: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#110] BYPASS (Orig -> Pure): {R:284, G:2bc, B:300} -> Expected:{r:'h284, g:'h2bc, b:'h300, bypass:'h1, offset_val:'h14} == Actual:{r:'h284, g:'h2bc, b:'h300, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 15910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#111] BYPASS (Orig -> Pure): {R:274, G:091, B:3f0} -> Expected:{r:'h274, g:'h91, b:'h3f0, bypass:'h1, offset_val:'h14} == Actual:{r:'h274, g:'h91, b:'h3f0, bypass:'h1, offset_val:'h14}
```

```
UVM_INFO lb_ro_sb.sv(101) @ 19670: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 19670: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM hact*vact*frame 수= sb [lb_ro_sb_c] - Total Input Pixels : 128
UVM 8 * 4 * 4 = 128 sb [lb_ro_sb_c] - Total Output Pixels : 128
UVM sb [lb_ro_sb_c] - Match Count : 128
UVM sb [lb_ro_sb_c] - Mismatch Count : 0
UVM sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(107) @ 19670: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(108) @ 19670: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ; //INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10 ;
    fix_g_data     == 10 ;
    fix_b_data     == 10 ;
    user_bypass    == 1 ;
    user_offset_val == 20 ;
    user_vsw       == 4 ;
    user_vbp       == 2 ;
    user_vact      == 4 ;
    user_vfp       == 2 ;
    user_hsw       == 2 ;
    user_hbp       == 4 ;
    user_hact      == 8 ;
    user_hfp       == 4 ;
})
```

시나리오별 Report 결과

2) Report - Scenario1 - 다양한 frame size에서 검증

```
UVM_INFO lb_ro_seq.lib.sv(91) @ 67330: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 71110: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1601] BYPASS (Orig -> Pure): {R:20c, G:1c9, B:00c} -> Expected:{r:'h20c, g:'h1c9, b:'hc, bypass:'h1, offset_val:'h14} == Actual:{r:'h20c, g:'h1c9, b:'hc, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71130: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1602] BYPASS (Orig -> Pure): {R:3ba, G:1e2, B:12f} -> Expected:{r:'h3ba, g:'h1e2, b:'h12f, bypass:'h1, offset_val:'h14} == Actual:{r:'h3ba, g:'h1e2, b:'h12f, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71150: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1603] BYPASS (Orig -> Pure): {R:1d7, G:30b, B:3a1} -> Expected:{r:'h1d7, g:'h30b, b:'h3a1, bypass:'h1, offset_val:'h14} == Actual:{r:'h1d7, g:'h30b, b:'h3a1, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71170: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1604] BYPASS (Orig -> Pure): {R:284, G:09b, B:162} -> Expected:{r:'h284, g:'h9b, b:'h162, bypass:'h1, offset_val:'h14} == Actual:{r:'h284, g:'h9b, b:'h162, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71190: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1605] BYPASS (Orig -> Pure): {R:05b, G:232, B:1c4} -> Expected:{r:'h5b, g:'h232, b:'h1c4, bypass:'h1, offset_val:'h14} == Actual:{r:'h5b, g:'h232, b:'h1c4, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71210: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1606] BYPASS (Orig -> Pure): {R:142, G:0e2, B:323} -> Expected:{r:'h142, g:'he2, b:'h323, bypass:'h1, offset_val:'h14} == Actual:{r:'h142, g:'he2, b:'h323, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1607] BYPASS (Orig -> Pure): {R:025, G:2a3, B:112} -> Expected:{r:'h25, g:'h2a3, b:'h112, bypass:'h1, offset_val:'h14} == Actual:{r:'h25, g:'h2a3, b:'h112, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71250: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1608] BYPASS (Orig -> Pure): {R:06d, G:32c, B:2f8} -> Expected:{r:'h6d, g:'h32c, b:'h2f8, bypass:'h1, offset_val:'h14} == Actual:{r:'h6d, g:'h32c, b:'h2f8, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71270: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1609] BYPASS (Orig -> Pure): {R:079, G:3ae, B:1c6} -> Expected:{r:'h79, g:'h3ae, b:'h1c6, bypass:'h1, offset_val:'h14} == Actual:{r:'h79, g:'h3ae, b:'h1c6, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71290: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1610] BYPASS (Orig -> Pure): {R:37d, G:35a, B:054} -> Expected:{r:'h37d, g:'h35a, b:'h54, bypass:'h1, offset_val:'h14} == Actual:{r:'h37d, g:'h35a, b:'h54, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71310: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1611] BYPASS (Orig -> Pure): {R:171, G:096, B:086} -> Expected:{r:'h171, g:'h96, b:'h86, bypass:'h1, offset_val:'h14} == Actual:{r:'h171, g:'h96, b:'h86, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1612] BYPASS (Orig -> Pure): {R:07d, G:15f, B:043} -> Expected:{r:'h7d, g:'h15f, b:'h43, bypass:'h1, offset_val:'h14} == Actual:{r:'h7d, g:'h15f, b:'h43, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71350: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1613] BYPASS (Orig -> Pure): {R:175, G:211, B:3ee} -> Expected:{r:'h175, g:'h211, b:'h3ee, bypass:'h1, offset_val:'h14} == Actual:{r:'h175, g:'h211, b:'h3ee, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71370: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1614] BYPASS (Orig -> Pure): {R:35c, G:1f0, B:0d5} -> Expected:{r:'h35c, g:'h1f0, b:'hd5, bypass:'h1, offset_val:'h14} == Actual:{r:'h35c, g:'h1f0, b:'hd5, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71390: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1615] BYPASS (Orig -> Pure): {R:3da, G:030, B:0dc} -> Expected:{r:'h3da, g:'h30, b:'hdc, bypass:'h1, offset_val:'h14} == Actual:{r:'h3da, g:'h30, b:'hdc, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 71410: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#1616] BYPASS (Orig -> Pure): {R:2be, G:310, B:14a} -> Expected:{r:'h2be, g:'h310, b:'h14a, bypass:'h1, offset_val:'h14} == Actual:{r:'h2be, g:'h310, b:'h14a, bypass:'h1, offset_val:'h14}
```

```
UVM_INFO lb_ro_sb.sv(101) @ 87830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 87830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM hact*vact*frame 수= b [lb_ro_sb_c] - Total Input Pixels : 2000
UVM 20 * 20 * 5 = 2000 b [lb_ro_sb_c] - Total Output Pixels : 2000
UVM b [lb_ro_sb_c] - Match Count : 2000
UVM b [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 87830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 87830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ;//INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10      ;
    fix_g_data     == 10      ;
    fix_b_data     == 10      ;
    user_bypass    == 1       ;
    user_offset_val == 20      ;
    user_vsw       == 4       ;
    user_vbp       == 2       ;
    user_vact      == 20      ;
    user_vfp       == 2       ;
    user_hsw       == 2       ;
    user_hbp       == 4       ;
    user_hact      == 20      ;
    user_hfp       == 4       ;
}
```

시나리오별 Report 결과

2) Report - Scenario2 - Random

```
UVM_INFO lb_ro_seq_lib.sv(91) @ 11570: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 13310: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#37] BYPASS (Orig -> Pure): {R:0a4, G:219, B:318} -> Expected:{r:'ha4, g:'h219, b:'h318, bypass:'h1, offset_val:'h14} == Actual:{r:'ha4, g:'h219, b:'h318, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#38] BYPASS (Orig -> Pure): {R:19f, G:011, B:203} -> Expected:{r:'h19f, g:'h11, b:'h203, bypass:'h1, offset_val:'h14} == Actual:{r:'h19f, g:'h11, b:'h203, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13350: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#39] BYPASS (Orig -> Pure): {R:23a, G:3b8, B:138} -> Expected:{r:'h23a, g:'h3b8, b:'h138, bypass:'h1, offset_val:'h14} == Actual:{r:'h23a, g:'h3b8, b:'h138, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13370: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#40] BYPASS (Orig -> Pure): {R:h94, G:188, B:0dc} -> Expected:{r:'h94, g:'h188, b:'hdc, bypass:'h1, offset_val:'h14} == Actual:{r:'h94, g:'h188, b:'hdc, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13390: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#41] BYPASS (Orig -> Pure): {R:h55, G:0cf, B:3b7} -> Expected:{r:'h55, g:'hcf, b:'h3b7, bypass:'h1, offset_val:'h14} == Actual:{r:'h55, g:'hcf, b:'h3b7, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13410: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#42] BYPASS (Orig -> Pure): {R:h18b, G:387, B:2a7} -> Expected:{r:'h18b, g:'h387, b:'h2a7, bypass:'h1, offset_val:'h14} == Actual:{r:'h18b, g:'h387, b:'h2a7, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#43] BYPASS (Orig -> Pure): {R:0a4, G:2db, B:1ee} -> Expected:{r:'ha4, g:'h2db, b:'h1ee, bypass:'h1, offset_val:'h14} == Actual:{r:'ha4, g:'h2db, b:'h1ee, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13850: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#44] BYPASS (Orig -> Pure): {R:172, G:215, B:344} -> Expected:{r:'h172, g:'h215, b:'h344, bypass:'h1, offset_val:'h14} == Actual:{r:'h172, g:'h215, b:'h344, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13870: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#45] BYPASS (Orig -> Pure): {R:14f, G:316, B:26f} -> Expected:{r:'h14f, g:'h316, b:'h26f, bypass:'h1, offset_val:'h14} == Actual:{r:'h14f, g:'h316, b:'h26f, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_seq_lib.sv(185) @ 14430: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming ends..
```

매 rising clock에서 rgb값을 random으로 생성

```
UVM_INFO lb_ro_sb.sv(101) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM_INFO lb_ro_sb.sv(103) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 45
UVM_INFO lb_ro_sb.sv(104) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 45
UVM_INFO lb_ro_sb.sv(105) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 45
UVM_INFO lb_ro_sb.sv(106) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ; //INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10 ;
    fix_g_data     == 10 ;
    fix_b_data     == 10 ;
    user_bypass    == 1 ;
    user_offset_val == 20 ;
    user_vsw       == 4 ;
    user_vbp       == 2 ;
    user_vact       == 3 ;
    user_vfp       == 2 ;
    user_hsw       == 2 ;
    user_hbp       == 4 ;
    user_hact       == 3 ;
    user_hfp       == 4 ;
})
```

data_mode,fix_data를 제외한 모든 parameter값은 동일하게 설정

시나리오별 Report 결과

2) Report - Scenario2 - FIX

```
UVM_INFO lb_ro_seq_lib.sv(91) @ 8710: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 10450: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#28] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10470: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#29] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10490: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#30] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10710: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#31] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10730: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#32] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10750: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#33] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10970: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#34] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 10990: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#35] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 11010: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#36] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_seq_lib.sv(185) @ 11570: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming ends..
```

모든 값이 10으로 고정

```
UVM_INFO lb_ro_sb.sv(101) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM_INFO lb_ro_sb.sv(103) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 45
UVM_INFO lb_ro_sb.sv(104) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 45
UVM_INFO lb_ro_sb.sv(105) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 45
UVM_INFO lb_ro_sb.sv(106) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seq,
{
    data_mode      == FIX      ; // INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10       ;
    fix_g_data     == 10       ;
    fix_b_data     == 10       ;
    user_bypass    == 1        ;
    user_offset_val == 20       ;
    user_vsw       == 4        ;
    user_vbp       == 2        ;
    user_vact      == 3        ;
    user_vfp       == 2        ;
    user_hsw       == 2        ;
    user_hbp       == 4        ;
    user_hact      == 3        ;
    user_hfp       == 4        ;
})
```

시나리오별 Report 결과

2) Report - Scenario2 - Increase

```
UVM_INFO lb_ro_seq_lib.sv(91) @ 8710: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..  
UVM_INFO lb_ro_sb.sv(88) @ 10450: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#28] BYPASS (Orig -> Pure): {R:000, G:000, B:000} -> Expected:{r:'h0, g:'h0, b:'h0, bypass:'h1, offset_val:'h14} == Actual:{r:'h0, g:'h0, b:'h0, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10470: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#29] BYPASS (Orig -> Pure): {R:005, G:005, B:005} -> Expected:{r:'h5, g:'h5, b:'h5, bypass:'h1, offset_val:'h14} == Actual:{r:'h5, g:'h5, b:'h5, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10490: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#30] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10510: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#31] BYPASS (Orig -> Pure): {R:00f, G:00f, B:00f} -> Expected:{r:'hf, g:'hf, b:'hf, bypass:'h1, offset_val:'h14} == Actual:{r:'hf, g:'hf, b:'hf, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10530: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#32] BYPASS (Orig -> Pure): {R:014, G:014, B:014} -> Expected:{r:'h14, g:'h14, b:'h14, bypass:'h1, offset_val:'h14} == Actual:{r:'h14, g:'h14, b:'h14, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10550: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#33] BYPASS (Orig -> Pure): {R:019, G:019, B:019} -> Expected:{r:'h19, g:'h19, b:'h19, bypass:'h1, offset_val:'h14} == Actual:{r:'h19, g:'h19, b:'h19, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10970: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#34] BYPASS (Orig -> Pure): {R:01e, G:01e, B:01e} -> Expected:{r:'h1e, g:'h1e, b:'h1e, bypass:'h1, offset_val:'h14} == Actual:{r:'h1e, g:'h1e, b:'h1e, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 10990: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#35] BYPASS (Orig -> Pure): {R:023, G:023, B:023} -> Expected:{r:'h23, g:'h23, b:'h23, bypass:'h1, offset_val:'h14} == Actual:{r:'h23, g:'h23, b:'h23, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 11010: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#36] BYPASS (Orig -> Pure): {R:028, G:028, B:028} -> Expected:{r:'h28, g:'h28, b:'h28, bypass:'h1, offset_val:'h14} == Actual:{r:'h28, g:'h28, b:'h28, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_seq_lib.sv(185) @ 11570: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming ends..
```

매 rising clock에서 5씩 더해짐

```
UVM_INFO lb_ro_seq_lib.sv(91) @ 8710: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..  
UVM_INFO lb_ro_sb.sv(88) @ 13570: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#40] BYPASS (Orig -> Pure): {R:000, G:000, B:000} -> Expected:{r:'h0, g:'h0, b:'h0, bypass:'h1, offset_val:'h14} == Actual:{r:'h0, g:'h0, b:'h0, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13590: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#41] BYPASS (Orig -> Pure): {R:005, G:005, B:005} -> Expected:{r:'h5, g:'h5, b:'h5, bypass:'h1, offset_val:'h14} == Actual:{r:'h5, g:'h5, b:'h5, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13610: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#42] BYPASS (Orig -> Pure): {R:00a, G:00a, B:00a} -> Expected:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14} == Actual:{r:'ha, g:'ha, b:'ha, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13570: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#40] BYPASS (Orig -> Pure): {R:00f, G:00f, B:00f} -> Expected:{r:'hf, g:'hf, b:'hf, bypass:'h1, offset_val:'h14} == Actual:{r:'hf, g:'hf, b:'hf, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13590: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#41] BYPASS (Orig -> Pure): {R:014, G:014, B:014} -> Expected:{r:'h14, g:'h14, b:'h14, bypass:'h1, offset_val:'h14} == Actual:{r:'h14, g:'h14, b:'h14, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13610: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#42] BYPASS (Orig -> Pure): {R:019, G:019, B:019} -> Expected:{r:'h19, g:'h19, b:'h19, bypass:'h1, offset_val:'h14} == Actual:{r:'h19, g:'h19, b:'h19, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13830: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#43] BYPASS (Orig -> Pure): {R:01e, G:01e, B:01e} -> Expected:{r:'h1e, g:'h1e, b:'h1e, bypass:'h1, offset_val:'h14} == Actual:{r:'h1e, g:'h1e, b:'h1e, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13850: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#44] BYPASS (Orig -> Pure): {R:023, G:023, B:023} -> Expected:{r:'h23, g:'h23, b:'h23, bypass:'h1, offset_val:'h14} == Actual:{r:'h23, g:'h23, b:'h23, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_sb.sv(88) @ 13870: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#45] BYPASS (Orig -> Pure): {R:028, G:028, B:028} -> Expected:{r:'h28, g:'h28, b:'h28, bypass:'h1, offset_val:'h14} == Actual:{r:'h28, g:'h28, b:'h28, bypass:'h1, offset_val:'h14}  
UVM_INFO lb_ro_seq_lib.sv(185) @ 14430: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming ends..
```

매 frame 시작시 0으로 변경됨

```
UVM_INFO lb_ro_sb.sv(101) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====  
UVM_INFO lb_ro_sb.sv(102) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT  
UVM_INFO lb_ro_sb.sv(103) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 45  
UVM_INFO lb_ro_sb.sv(104) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 45  
UVM_INFO lb_ro_sb.sv(105) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 45  
UVM_INFO lb_ro_sb.sv(106) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0  
UVM_INFO lb_ro_sb.sv(107) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0  
UVM INFO lb_ro_sb.sv(108) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
`uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,  
{  
    data_mode      == INCREASE ; //INCREASE ; // RANDOM, FIX, INCREASE  
    fix_r_data     == 10      ;  
    fix_g_data     == 10      ;  
    fix_b_data     == 10      ;  
    user_bypass    == 1       ;  
    user_offset_val == 20      ;  
    user_vsw       == 4       ;  
    user_vbp       == 2       ;  
    user_vact       == 3       ;  
    user_vfp       == 2       ;  
    user_hsw       == 2       ;  
    user_hbp       == 4       ;  
    user_hact       == 3       ;  
    user_hfp       == 4       ;  
}
```


시나리오별 Report 결과

3) Report - Scenario3 – Bypass, offset

```
JVM_INFO lb_ro_seq.lib.sv(91) @ 8710: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
JVM_INFO lb_ro_sb.sv(88) @ 10450: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#28] BYPASS (Orig -> Pure): {R:1d4, G:358, B:312} -> Expected:{r:'h1d4, g:'h358, b:'h312, bypass:'h1, offset_val:'h14} == Actual:{r:'h1d4, g:'h358, b:'h312, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10470: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#29] BYPASS (Orig -> Pure): {R:09a, G:28b, B:22e} -> Expected:{r:'h9a, g:'h28b, b:'h22e, bypass:'h1, offset_val:'h14} == Actual:{r:'h9a, g:'h28b, b:'h22e, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10490: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#30] BYPASS (Orig -> Pure): {R:032, G:206, B:169} -> Expected:{r:'h32, g:'h206, b:'h169, bypass:'h1, offset_val:'h14} == Actual:{r:'h32, g:'h206, b:'h169, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10710: 들어온 데이터가 그대로 출력됨 {R:246, G:2b4, B:06d} -> Expected:{r:'h246, g:'h2b4, b:'h6d, bypass:'h1, offset_val:'h14} == Actual:{r:'h246, g:'h2b4, b:'h6d, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10730: {R:3e5, G:3e0, B:1d7} -> Expected:{r:'h3e5, g:'h3e0, b:'h1d7, bypass:'h1, offset_val:'h14} == Actual:{r:'h3e5, g:'h3e0, b:'h1d7, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10750: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#33] BYPASS (Orig -> Pure): {R:17d, G:309, B:019} -> Expected:{r:'h17d, g:'h309, b:'h19, bypass:'h1, offset_val:'h14} == Actual:{r:'h17d, g:'h309, b:'h19, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10970: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#34] BYPASS (Orig -> Pure): {R:0a7, G:021, B:33a} -> Expected:{r:'ha7, g:'h21, b:'h33a, bypass:'h1, offset_val:'h14} == Actual:{r:'ha7, g:'h21, b:'h33a, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 10990: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#35] BYPASS (Orig -> Pure): {R:09a, G:109, B:3a8} -> Expected:{r:'h9a, g:'h109, b:'h3a8, bypass:'h1, offset_val:'h14} == Actual:{r:'h9a, g:'h109, b:'h3a8, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_sb.sv(88) @ 11010: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#36] BYPASS (Orig -> Pure): {R:3d6, G:16b, B:316} -> Expected:{r:'h3d6, g:'h16b, b:'h316, bypass:'h1, offset_val:'h14} == Actual:{r:'h3d6, g:'h16b, b:'h316, bypass:'h1, offset_val:'h14}
JVM_INFO lb_ro_seq.lib.sv(185) @ 11570: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming ends..
```

```
UVM_INFO lb_ro_sb.sv(101) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM_INFO lb_ro_sb.sv(103) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 45
UVM_INFO lb_ro_sb.sv(104) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 45
UVM_INFO lb_ro_sb.sv(105) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 45
UVM_INFO lb_ro_sb.sv(106) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ; //INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10      ;
    fix_g_data     == 10      ;
    fix_b_data     == 10      ;
    user_bypass    == 1       ;
    user_offset_val == 20      ;
    user_vsw       == 4       ;
    user_vbp       == 2       ;
    user_vact       == 3       ;
    user_vfp       == 2       ;
    user_hsw       == 2       ;
    user_hbp       == 4       ;
    user_hact       == 3       ;
    user_hfp       == 4       ;
})
```


시나리오별 Report 결과

3) Report - Scenario3 – Bypass, offset

```
UVM_INFO lb_ro_seq.lib.sv(91) @ 11570: uvm_test_top.tb.lb_ro_env.lb_ro_agent.lb_ro_sequencer@@video.lb_ro_vseq.lb_ro_user_seq [lb_ro_user_seq_c] vtiming starts..
UVM_INFO lb_ro_sb.sv(88) @ 13590: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#37] OFFSET (Orig+Off): {R:0a4+014, G:219+014, B:318+014} -> Expected:{r:'hb8, g:'h22d, b:'h32c, bypass:'h0, offset_val:'h14} == Actual:{r:'hb8, g:'h22d, b:'h32c, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13610: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#38] OFFSET (Orig+Off): {R:19f+014, G:011+014, B:203+014} -> Expected:{r:'h1b3, g:'h25, b:'h217, bypass:'h0, offset_val:'h14} == Actual:{r:'h1b3, g:'h25, b:'h217, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 13630: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#39] OFFSET (Orig+Off): {R:23a+014, G:3b8+014, B:138+014} -> Expected:{r:'h24e, g:'h3cc, b:'h14c, bypass:'h0, offset_val:'h14} == Actual:{r:'h24e, g:'h3cc, b:'h14c, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14110: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#43] OFFSET (Orig+Off): {R:094+014, G:188+014, B:0dc+014} -> Expected:{r:'ha8, g:'h19c, b:'hf0, bypass:'h0, offset_val:'h14} == Actual:{r:'ha8, g:'h19c, b:'hf0, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14130: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#44] OFFSET (Orig+Off): {R:055+014, G:0cf+014, B:3b7+014} -> Expected:{r:'h69, g:'he3, b:'h3cb, bypass:'h0, offset_val:'h14} == Actual:{r:'h69, g:'he3, b:'h3cb, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14150: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#45] OFFSET (Orig+Off): {R:18b+014, G:387+014, B:2a7+014} -> Expected:{r:'h19f, g:'h39b, b:'h2bb, bypass:'h0, offset_val:'h14} == Actual:{r:'h19f, g:'h39b, b:'h2bb, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14110: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#43] OFFSET (Orig+Off): {R:0a4+014, G:2db+014, B:1ee+014} -> Expected:{r:'hb8, g:'h2ef, b:'h202, bypass:'h0, offset_val:'h14} == Actual:{r:'hb8, g:'h2ef, b:'h202, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14130: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#44] OFFSET (Orig+Off): {R:172+014, G:215+014, B:344+014} -> Expected:{r:'h186, g:'h229, b:'h358, bypass:'h0, offset_val:'h14} == Actual:{r:'h186, g:'h229, b:'h358, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 14150: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#45] OFFSET (Orig+Off): {R:14f+014, G:316+014, B:26f+014} -> Expected:{r:'h163, g:'h32a, b:'h283, bypass:'h0, offset_val:'h14} == Actual:{r:'h163, g:'h32a, b:'h283, bypass:'h0, offset_val:'h14}
```

offset_value와 들어온 데이터를 더해서 출력값과 비교함

```
UVM_INFO lb_ro_sb.sv(101) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM_INFO lb_ro_sb.sv(103) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 45
UVM_INFO lb_ro_sb.sv(104) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 45
UVM_INFO lb_ro_sb.sv(105) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 45
UVM_INFO lb_ro_sb.sv(106) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 16090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

```
uvm_do_on_with(lb_ro_vseq, p_sequencer.lb_ro_seqr,
{
    data_mode      == RANDOM ; // INCREASE ; // RANDOM, FIX, INCREASE
    fix_r_data     == 10      ;
    fix_g_data     == 10      ;
    fix_b_data     == 10      ;
    user_bypass    == 1       ;
    user_offset_val == 20      ;
    user_vsw       == 4       ;
    user_vbp       == 2       ;
    user_vact       == 3       ;
    user_vfp       == 2       ;
    user_hsw       == 2       ;
    user_hbp       == 4       ;
    user_hact       == 3       ;
    user_hfp       == 4       ;
})
```

시나리오별 Report 결과

4) Report - 추가사항 – pixel 단위

```
UVM_INFO lb_ro_sb.sv(88) @ 22330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#241] OFFSET (Orig+Off): {R:299+014, G:126+014, B:2ec+014} -> Expected:{r:'h2ad, g:'h13a, b:'h300, bypass:'h0, offset_val:'h14} == Actual:{r:'h2ad, g:'h13a, b:'h300, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22350: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#242] OFFSET (Orig+Off): {R:323+014, G:324+014, B:245+014} -> Expected:{r:'h337, g:'h338, b:'h259, bypass:'h0, offset_val:'h14} == Actual:{r:'h337, g:'h338, b:'h259, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22370: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#243] OFFSET (Orig+Off): {R:318+014, G:3f9+014, B:3d2+014} -> Expected:{r:'h32c, g:'h3ff, b:'h3e6, bypass:'h0, offset_val:'h14} == Actual:{r:'h32c, g:'h3ff, b:'h3e6, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22390: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#244] OFFSET (Orig+Off): {R:347+014, G:00f+014, B:136+014} -> Expected:{r:'h35b, g:'h23, b:'h14a, bypass:'h0, offset_val:'h14} == Actual:{r:'h35b, g:'h23, b:'h14a, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22410: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#245] OFFSET (Orig+Off): {R:24f+014, G:362+014, B:216+014} -> Expected:{r:'h263, g:'h376, b:'h22a, bypass:'h0, offset_val:'h14} == Actual:{r:'h263, g:'h376, b:'h22a, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22430: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#246] OFFSET (Orig+Off): {R:0e6+014, G:19c+014, B:1a2+014} -> Expected:{r:'hfa, g:'h1b0, b:'h1b6, bypass:'h0, offset_val:'h14} == Actual:{r:'hfa, g:'h1b0, b:'h1b6, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22450: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#247] OFFSET (Orig+Off): {R:0a8+014, G:1fe+014, B:116+014} -> Expected:{r:'hbc, g:'h212, b:'h12a, bypass:'h0, offset_val:'h14} == Actual:{r:'hbc, g:'h212, b:'h12a, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22470: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#248] OFFSET (Orig+Off): {R:290+014, G:2f1+014, B:320+014} -> Expected:{r:'h2a4, g:'h305, b:'h334, bypass:'h0, offset_val:'h14} == Actual:{r:'h2a4, g:'h305, b:'h334, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22490: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#249] OFFSET (Orig+Off): {R:29d+014, G:068+014, B:2ef+014} -> Expected:{r:'h2b1, g:'h7c, b:'h303, bypass:'h0, offset_val:'h14} == Actual:{r:'h2b1, g:'h7c, b:'h303, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22510: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#250] OFFSET (Orig+Off): {R:33c+014, G:251+014, B:256+014} -> Expected:{r:'h350, g:'h265, b:'h26a, bypass:'h0, offset_val:'h14} == Actual:{r:'h350, g:'h265, b:'h26a, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22730: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#251] OFFSET (Orig+Off): {R:377+014, G:1bc+014, B:149+014} -> Expected:{r:'h377, g:'h1bc, b:'h149, bypass:'h0, offset_val:'h14} == Actual:{r:'h377, g:'h1bc, b:'h149, bypass:'h0, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#252] BYPASS (Orig -> Pure): {R:17e, G:20d, B:1a0} -> Expected:{r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14} == Actual:{r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21930: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#253] BYPASS (Orig -> Pure): {R:36c, G:016, B:0e2} -> Expected:{r:'h36c, g:'h16, b:'he2, bypass:'h1, offset_val:'h14} == Actual:{r:'h36c, g:'h16, b:'he2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21950: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#254] BYPASS (Orig -> Pure): {R:323, G:324, B:245} -> Expected:{r:'h323, g:'h324, b:'h245, bypass:'h1, offset_val:'h14} == Actual:{r:'h323, g:'h324, b:'h245, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21970: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#255] BYPASS (Orig -> Pure): {R:318, G:3f9, B:3d2} -> Expected:{r:'h318, g:'h3f9, b:'h3d2, bypass:'h1, offset_val:'h14} == Actual:{r:'h318, g:'h3f9, b:'h3d2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 21990: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#256] BYPASS (Orig -> Pure): {R:347, G:00f, B:136} -> Expected:{r:'h347, g:'hf, b:'h136, bypass:'h1, offset_val:'h14} == Actual:{r:'h347, g:'hf, b:'h136, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22010: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#257] BYPASS (Orig -> Pure): {R:24f, G:362, B:216} -> Expected:{r:'h24f, g:'h362, b:'h216, bypass:'h1, offset_val:'h14} == Actual:{r:'h24f, g:'h362, b:'h216, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22030: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#258] BYPASS (Orig -> Pure): {R:0e6, G:19c, B:1a2} -> Expected:{r:'he6, g:'h19c, b:'h1a2, bypass:'h1, offset_val:'h14} == Actual:{r:'he6, g:'h19c, b:'h1a2, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22050: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#259] BYPASS (Orig -> Pure): {R:0a8, G:1fe, B:116} -> Expected:{r:'ha8, g:'h1fe, b:'h116, bypass:'h1, offset_val:'h14} == Actual:{r:'ha8, g:'h1fe, b:'h116, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22070: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#260] BYPASS (Orig -> Pure): {R:290, G:2f1, B:320} -> Expected:{r:'h290, g:'h2f1, b:'h320, bypass:'h1, offset_val:'h14} == Actual:{r:'h290, g:'h2f1, b:'h320, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22090: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#261] BYPASS (Orig -> Pure): {R:29d, G:068, B:2ef} -> Expected:{r:'h29d, g:'h68, b:'h2ef, bypass:'h1, offset_val:'h14} == Actual:{r:'h29d, g:'h68, b:'h2ef, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22310: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#262] BYPASS (Orig -> Pure): {R:33c, G:251, B:256} -> Expected:{r:'h33c, g:'h251, b:'h256, bypass:'h1, offset_val:'h14} == Actual:{r:'h33c, g:'h251, b:'h256, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#263] BYPASS (Orig -> Pure): {R:377, G:1bc, B:149} -> Expected:{r:'h377, g:'h1bc, b:'h149, bypass:'h1, offset_val:'h14} == Actual:{r:'h377, g:'h1bc, b:'h149, bypass:'h1, offset_val:'h14}
UVM_INFO lb_ro_sb.sv(88) @ 22350: uvm_test_top.tb.lb_ro_env.lb_ro_sb [DATA_CMP] MATCH SUCCESS! [Pkt#264] BYPASS (Orig -> Pure): {R:17e, G:20d, B:1a0} -> Expected:{r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14} == Actual:{r:'h17e, g:'h20d, b:'h1a0, bypass:'h1, offset_val:'h14}
```

rising clk 마다 match

```
UVM_INFO lb_ro_sb.sv(101) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
UVM_INFO lb_ro_sb.sv(102) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] FINAL REPORT
UVM_INFO lb_ro_sb.sv(103) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Input Pixels : 320
UVM_INFO lb_ro_sb.sv(104) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Total Output Pixels : 320
UVM_INFO lb_ro_sb.sv(105) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Match Count : 320
UVM_INFO lb_ro_sb.sv(106) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Mismatch Count : 0
UVM_INFO lb_ro_sb.sv(107) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] - Remaining In Q : 0
UVM_INFO lb_ro_sb.sv(108) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] =====
```

시나리오별 Report 결과

4) Report - 추가사항 - line 단위

```
UVM_INFO lb_ro_sb.sv(36) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
UVM_INFO lb_ro_sb.sv(37) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] Time: 25330 | Mode: OFFSET(Val: 20) | Total Pixels: 10
UVM_INFO lb_ro_sb.sv(38) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 0] R:[0d0 + 20 = 0e4 | Act: 0e4] G:[38d + 20 = 3a1 | Act: 3a1] B:[32c + 20 = 340 | Act: 340]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 1] R:[0b5 + 20 = 0c9 | Act: 0c9] G:[2ff + 20 = 313 | Act: 313] B:[245 + 20 = 259 | Act: 259]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 2] R:[0c6 + 20 = 0da | Act: 0da] G:[1f0 + 20 = 204 | Act: 204] B:[0b4 + 20 = 0c8 | Act: 0c8]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 3] R:[23b + 20 = 24f | Act: 24f] G:[2f3 + 20 = 307 | Act: 307] B:[36c + 20 = 380 | Act: 380]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 4] R:[18a + 20 = 19e | Act: 19e] G:[3ba + 20 = 3ce | Act: 3ce] B:[059 + 20 = 06d | Act: 06d]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 5] R:[381 + 20 = 395 | Act: 395] G:[2a8 + 20 = 2bc | Act: 2bc] B:[3e2 + 20 = 3f6 | Act: 3f6]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 6] R:[272 + 20 = 286 | Act: 286] G:[14f + 20 = 163 | Act: 163] B:[273 + 20 = 287 | Act: 287]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 7] R:[0b7 + 20 = 0cb | Act: 0cb] G:[020 + 20 = 034 | Act: 034] B:[080 + 20 = 094 | Act: 094]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 8] R:[1ac + 20 = 1c0 | Act: 1c0] G:[00e + 20 = 022 | Act: 022] B:[23a + 20 = 24e | Act: 24e]
UVM_INFO lb_ro_sb.sv(64) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 9] R:[080 + 20 = 094 | Act: 094] B:[0f1 + 20 = 105 | Act: 105]
UVM_INFO lb_ro_sb.sv(75) @ 25330: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
```

동일한 clk에서 line에 저장된 값들을 match

```
UVM_INFO lb_ro_sb.sv(36) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
UVM_INFO lb_ro_sb.sv(37) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] Time: 24910 | Mode: BYPASS | Total Pixels: 10
UVM_INFO lb_ro_sb.sv(38) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 0] R:[Exp:0d0|Act:0d0] G:[Exp:38d|Act:38d] B:[Exp:32c|Act:32c]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 1] R:[Exp:0b5|Act:0b5] G:[Exp:2ff|Act:2ff] B:[Exp:245|Act:245]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 2] R:[Exp:0c6|Act:0c6] G:[Exp:1f0|Act:1f0] B:[Exp:0b4|Act:0b4]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 3] R:[Exp:23b|Act:23b] G:[Exp:2f3|Act:2f3] B:[Exp:36c|Act:36c]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 4] R:[Exp:18a|Act:18a] G:[Exp:3ba|Act:3ba] B:[Exp:059|Act:059]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 5] R:[Exp:381|Act:381] G:[Exp:2a8|Act:2a8] B:[Exp:3e2|Act:3e2]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 6] R:[Exp:272|Act:272] G:[Exp:14f|Act:14f] B:[Exp:273|Act:273]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 7] R:[Exp:0b7|Act:0b7] G:[Exp:020|Act:020] B:[Exp:080|Act:080]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 8] R:[Exp:1ac|Act:1ac] G:[Exp:00e|Act:00e] B:[Exp:23a|Act:23a]
UVM_INFO lb_ro_sb.sv(64) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_MATCH] [Px1 9] R:[Exp:080|Act:080] G:[Exp:1fa|Act:1fa] B:[Exp:0f1|Act:0f1]
UVM_INFO lb_ro_sb.sv(75) @ 24910: uvm_test_top.tb.lb_ro_env.lb_ro_sb [SB_LINE] =====
```

```
UVM_INFO lb_ro_sb.sv(84) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] -----
UVM_INFO lb_ro_sb.sv(85) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] Pixel Match/Mismatch : 320 / 0
UVM_INFO lb_ro_sb.sv(86) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] Line Match/Mismatch : 32 / 0
UVM_INFO lb_ro_sb.sv(87) @ 28230: uvm_test_top.tb.lb_ro_env.lb_ro_sb [lb_ro_sb_c] -----
```




시나리오별 Report 결과

4) Report - 추가사항 – frame 단위

UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:0 [BYPASS] EXP:{196,1a5,06f} ACT:{196,1a5,06f}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:0 [OFFSET:20] ORG:{196,1a5,06f} -> EXP:{1aa,1b9,083} ACT:{1aa,1b9,083}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:1 [BYPASS] EXP:{13c,144,0c4} ACT:{13c,144,0c4}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:1 [OFFSET:20] ORG:{13c,144,0c4} -> EXP:{150,158,0d8} ACT:{150,158,0d8}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:2 [BYPASS] EXP:{0a0,1a6,287} ACT:{0a0,1a6,287}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:2 [OFFSET:20] ORG:{0a0,1a6,287} -> EXP:{0b4,1ba,29b} ACT:{0b4,1ba,29b}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:3 [BYPASS] EXP:{23a,053,0b9} ACT:{23a,053,0b9}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:3 [OFFSET:20] ORG:{23a,053,0b9} -> EXP:{24e,067,0cd} ACT:{24e,067,0cd}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:4 [BYPASS] EXP:{046,380,096} ACT:{046,380,096}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:4 [OFFSET:20] ORG:{046,380,096} -> EXP:{05a,394,0aa} ACT:{05a,394,0aa}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:5 [BYPASS] EXP:{19c,2aa,031} ACT:{19c,2aa,031}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:5 [OFFSET:20] ORG:{19c,2aa,031} -> EXP:{1b0,2be,045} ACT:{1b0,2be,045}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:6 [BYPASS] EXP:{25f,2cd,082} ACT:{25f,2cd,082}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:6 [OFFSET:20] ORG:{25f,2cd,082} -> EXP:{273,2e1,096} ACT:{273,2e1,096}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:7 [BYPASS] EXP:{0da,3b4,2de} ACT:{0da,3b4,2de}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:0 P:7 [OFFSET:20] ORG:{0da,3b4,2de} -> EXP:{0ee,3c8,2f2} ACT:{0ee,3c8,2f2}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:0 [BYPASS] EXP:{152,09f,093} ACT:{152,09f,093}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:0 [OFFSET:20] ORG:{152,09f,093} -> EXP:{166,0b3,0a7} ACT:{166,0b3,0a7}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:1 [BYPASS] EXP:{2a5,2f4,01f} ACT:{2a5,2f4,01f}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:1 [OFFSET:20] ORG:{2a5,2f4,01f} -> EXP:{2b9,308,033} ACT:{2b9,308,033}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:2 [BYPASS] EXP:{3f7,009,23b} ACT:{3f7,009,23b}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:2 [OFFSET:20] ORG:{3f7,009,23b} -> EXP:{3ff,01d,24f} ACT:{3ff,01d,24f}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:3 [BYPASS] EXP:{03d,25f,1ac} ACT:{03d,25f,1ac}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:3 [OFFSET:20] ORG:{03d,25f,1ac} -> EXP:{051,273,1c0} ACT:{051,273,1c0}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:4 [BYPASS] EXP:{2d4,097,3a0} ACT:{2d4,097,3a0}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:4 [OFFSET:20] ORG:{2d4,097,3a0} -> EXP:{2e8,0ab,3b4} ACT:{2e8,0ab,3b4}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:5 [BYPASS] EXP:{284,2bc,300} ACT:{284,2bc,300}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:5 [OFFSET:20] ORG:{284,2bc,300} -> EXP:{298,2d0,314} ACT:{298,2d0,314}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:6 [BYPASS] EXP:{274,091,3f0} ACT:{274,091,3f0}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:6 [OFFSET:20] ORG:{274,091,3f0} -> EXP:{288,0a5,3ff} ACT:{288,0a5,3ff}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:7 [BYPASS] EXP:{08c,089,188} ACT:{08c,089,188}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:1 P:7 [OFFSET:20] ORG:{08c,089,188} -> EXP:{0a0,09d,19c} ACT:{0a0,09d,19c}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:0 [BYPASS] EXP:{29a,1e0,262} ACT:{29a,1e0,262}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:0 [OFFSET:20] ORG:{29a,1e0,262} -> EXP:{2ae,1f4,276} ACT:{2ae,1f4,276}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:1 [BYPASS] EXP:{15e,17f,18e} ACT:{15e,17f,18e}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:1 [OFFSET:20] ORG:{15e,17f,18e} -> EXP:{172,193,1a2} ACT:{172,193,1a2}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:2 [BYPASS] EXP:{212,2d6,00d} ACT:{212,2d6,00d}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:2 [OFFSET:20] ORG:{212,2d6,00d} -> EXP:{226,2ea,021} ACT:{226,2ea,021}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:3 [BYPASS] EXP:{2c4,131,3e4} ACT:{2c4,131,3e4}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:3 [OFFSET:20] ORG:{2c4,131,3e4} -> EXP:{2d8,145,3f8} ACT:{2d8,145,3f8}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:4 [BYPASS] EXP:{2dc,24d,06b} ACT:{2dc,24d,06b}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:4 [OFFSET:20] ORG:{2dc,24d,06b} -> EXP:{2f0,261,07f} ACT:{2f0,261,07f}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:5 [BYPASS] EXP:{2c7,1ac,053} ACT:{2c7,1ac,053}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:5 [OFFSET:20] ORG:{2c7,1ac,053} -> EXP:{2db,1c0,067} ACT:{2db,1c0,067}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:6 [BYPASS] EXP:{273,342,0aa} ACT:{273,342,0aa}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:6 [OFFSET:20] ORG:{273,342,0aa} -> EXP:{287,356,0be} ACT:{287,356,0be}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:7 [BYPASS] EXP:{1af,1ec,0c1} ACT:{1af,1ec,0c1}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:2 P:7 [OFFSET:20] ORG:{1af,1ec,0c1} -> EXP:{1c3,200,0d5} ACT:{1c3,200,0d5}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:0 [BYPASS] EXP:{1f3,2f8,1b7} ACT:{1f3,2f8,1b7}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:0 [OFFSET:20] ORG:{1f3,2f8,1b7} -> EXP:{207,30c,1cb} ACT:{207,30c,1cb}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:1 [BYPASS] EXP:{23c,0c8,123} ACT:{23c,0c8,123}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:1 [OFFSET:20] ORG:{23c,0c8,123} -> EXP:{250,0dc,137} ACT:{250,0dc,137}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:2 [BYPASS] EXP:{1d6,32f,109} ACT:{1d6,32f,109}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:2 [OFFSET:20] ORG:{1d6,32f,109} -> EXP:{1ea,343,11d} ACT:{1ea,343,11d}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:3 [BYPASS] EXP:{1db,035,026} ACT:{1db,035,026}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:3 [OFFSET:20] ORG:{1db,035,026} -> EXP:{1ef,049,03a} ACT:{1ef,049,03a}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:4 [BYPASS] EXP:{112,2da,030} ACT:{112,2da,030}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:4 [OFFSET:20] ORG:{112,2da,030} -> EXP:{126,2ee,044} ACT:{126,2ee,044}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:5 [BYPASS] EXP:{298,35b,09e} ACT:{298,35b,09e}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:5 [OFFSET:20] ORG:{298,35b,09e} -> EXP:{2ac,36f,0b2} ACT:{2ac,36f,0b2}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:6 [BYPASS] EXP:{350,0fd,31b} ACT:{350,0fd,31b}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:6 [OFFSET:20] ORG:{350,0fd,31b} -> EXP:{364,111,32f} ACT:{364,111,32f}
UVM_INFO 1b_ro_sb.sv(95) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:7 [BYPASS] EXP:{08f,355,3ac} ACT:{08f,355,3ac}	UVM_INFO 1b_ro_sb.sv(104) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [DATA_VAL] L:3 P:7 [OFFSET:20] ORG:{08f,355,3ac} -> EXP:{0a3,369,3c0} ACT:{0a3,369,3c0}
UVM_INFO 1b_ro_sb.sv(125) @ 16670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [COMP_DONE] Frame Match Success!	UVM_INFO 1b_ro_sb.sv(125) @ 17050: uvm_test_top.tb.1b_ro_env.1b_ro_sb [COMP_DONE] Frame Match Success!

동일한 clk에서 frame에 저장된 값들을 match

```
UVM_INFO 1b_ro_sb.sv(138) @ 19670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [1b_ro_sb_c] =====
UVM_INFO 1b_ro_sb.sv(139) @ 19670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [1b_ro_sb_c] Total Pixel Match/Mismatch : 128 / 0
UVM_INFO 1b_ro_sb.sv(140) @ 19670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [1b_ro_sb_c] Total Line Match/Mismatch : 16 / 0
UVM_INFO 1b_ro_sb.sv(141) @ 19670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [1b_ro_sb_c] Total Frame Match/Mismatch : 4 / 0
UVM_INFO 1b_ro_sb.sv(142) @ 19670: uvm_test_top.tb.1b_ro_env.1b_ro_sb [1b_ro_sb_c] =====
```



7. Troubleshooting

07

Troubleshooting

#1. 재사용가능한 검증환경 구축:

- 문제상황 :
 - 현업에서는 훨씬 다양한 시나리오로, DUT를 검증
 - 시나리오가 늘어날수록, user_sequence에서 hard coding을 수행하는 방법은 비효율적
- 해결 과정 :
 - 각 시나리오별 class를 생성 후, base_sequence를 상속받고, generate_frame()만 시나리오에 맞게 수정하는 방식 활용
 - vseq_lib에서 각 시나리오별 class를 인스턴스만하면 재사용성 높은 Verification 환경 구축 가능

```
class lb_ro_random_seq_c extends lb_ro_base_seq_c;
virtual task generate_frame();

`uvm_info(get_type_name(), $sformatf("Frame: %0dx%0d (HACT=%0d, VACT=%0d)",
total_pixels, total_lines, hact, vact), UVM MEDIUM)

// 프레임 생성
for (int v=0; v<total_lines; v++) begin
// VSYNC 계산
vsync_val = (v < vsw) ? 1 : 0;

for (int h=0; h<total_pixels; h++) begin
// HSYNC 계산
hsync_val = (h < hsw) ? 1 : 0;

// DE (Data Enable) 계산
de_val = ((v >= vsw+vbp) && (v < vsw+vbp+vact) &&
(h >= hsw+hbp) && (h < hsw+hbp+hact)) ? 1 : 0;

// RANDOM: 매 클럭 urandom으로 RGB 생성
if (de_val) begin
r_val = $urandom_range(0, (1<<RGB_WIDTH)-1);
g_val = $urandom_range(0, (1<<RGB_WIDTH)-1);
b_val = $urandom_range(0, (1<<RGB_WIDTH)-1);
end else begin
r_val = 0;
g_val = 0;
b_val = 0;
end

send_signal(vsync_val, hsync_val, de_val, r_val, g_val, b_val);
end
endtask
endclass : lb_ro_random_seq_c
```

lb_ro_random_seq

```
class lb_ro_fix_seq_c extends lb_ro_base_seq_c;
virtual task generate_frame();

`uvm_info(get_type_name(), $sformatf("Frame: %0dx%0d (HACT=%0d, VACT=%0d)",
total_pixels, total_lines, hact, vact), UVM MEDIUM)

// 프레임 생성
for (int v=0; v<total_lines; v++) begin
vsync_val = (v < vsw) ? 1 : 0;

for (int h=0; h<total_pixels; h++) begin
hsync_val = (h < hsw) ? 1 : 0;

de_val = ((v >= vsw+vbp) && (v < vsw+vbp+vact) &&
(h >= hsw+hbp) && (h < hsw+hbp+hact)) ? 1 : 0;

// FIX: 고정된 RGB 값 사용
if (de_val) begin
r_val = fix_r_val;
g_val = fix_g_val;
b_val = fix_b_val;
end else begin
r_val = 0;
g_val = 0;
b_val = 0;
end

send_signal(vsync_val, hsync_val, de_val, r_val, g_val, b_val);
end
endtask
endclass : lb_ro_fix_seq_c
```

lb_ro_fix_seq

```
class lb_ro_increase_seq_c extends lb_ro_base_seq_c;
virtual task generate_frame();

// 카운터 초기화
r_counter = 0;
g_counter = 0;
b_counter = 0;
// 프레임 생성
for (int v=0; v<total_lines; v++) begin
vsync_val = (v < vsw) ? 1 : 0;

for (int h=0; h<total_pixels; h++) begin
hsync_val = (h < hsw) ? 1 : 0;

de_val = ((v >= vsw+vbp) && (v < vsw+vbp+vact) &&
(h >= hsw+hbp) && (h < hsw+hbp+hact)) ? 1 : 0;

// INCREASE: 0부터 시작하여 매 클럭 +1, MAX 도달 시 0으로 wrap
r_val = r_counter;
g_val = g_counter;
b_val = b_counter;

r_counter++;
if (r_counter >= (1<<RGB_WIDTH)) begin
r_counter = 0; // Wrap around
end

g_counter++;
if (g_counter >= (1<<RGB_WIDTH)) begin
g_counter = 0; // Wrap around
end

b_counter++;
if (b_counter >= (1<<RGB_WIDTH)) begin
b_counter = 0; // Wrap around
end

end else begin
r_val = 0;
g_val = 0;
b_val = 0;
end

send_signal(vsync_val, hsync_val, de_val, r_val, g_val, b_val);
end
endtask
endclass : lb_ro_increase_seq_c
```

lb_ro_increase_seq

#2. seq_lib에서 sync Data 생성이 제한되는 문제 :

- 문제상황 :

1. seq_lib에서 직접 clk 기반 delay를 사용할 수 없어 sync Data 생성이 제한되는 문제

- 해결 과정:

- 1) @(posedge p_sequencer.lb_ro_vif.i_clk) 형태로 사용하면 sequence 내부에서도 clock edge 기반 동기 제어가 가능함을 확인

→ 추가 문제 발생 (단순히 코드만 추가하면 i_clk 접근이 불가능)

추가 원인 분석

- 1) seq_lib는 직접 interface를 소유하지 않음.

- 2) env → agent → sequencer 경로로 Virtual Interface 연결이 필요했고, 해당 연결이 되어 있지 않아 vif가 null 상태로 존재.

- 2) lb_ro_sequencer를 factory 기반으로 생성 후

(lb_ro_sequencer_c::type_id::create("lb_ro_sequencer", this))

- 3) connect_phase()에서 env - agent - sequencer 간 interface를 명시적으로 연결.

- 4) 이를 통해 sequence에서 p_sequencer.lb_ro_vif.i_clk에 접근 가능하도록 구성.

#3. line, frame 단위의 데이터도 저장:

- 문제상황:

1. 기존 Scoreboard 구조의 Queue에서 line, frame 단위의 데이터도 저장하도록 하는 과정에서 문제 발생

- 해결 과정:

- 1) scoreboard 는 비교/검증 역할에 집중해야 하며 대용량 데이터의 저장 공간으로 사용하는 것은 부적절

→ 실제 데이터 단위를 정의하는 위치는 transaction (transfer.sv) 가 되어야 함을 확인.

- 2) 따라서 다음과 같이 queue 기반 멤버 변수를 정의하여 transfer.sv 내에서 line/frame 단위로 저장할 수 있게 함.

```
bit [RGB_WIDTH-1:0] r_payload[$];
```

```
bit [RGB_WIDTH-1:0] g_payload[$];
```

```
bit [RGB_WIDTH-1:0] b_payload[$];
```

- 3) 이후 field macro를 사용하여 queue 멤버 등록

```
`uvm_field_queue_int(r_payload, UVM_DEFAULT)
```

```
`uvm_field_queue_int(g_payload, UVM_DEFAULT)
```

```
`uvm_field_queue_int(b_payload, UVM_DEFAULT)
```

- 4) 이를 통해 transaction 내부에서 동적 공간 할당이 가능하도록 구성

#4. 비교 결과의 mismatch

- 문제 상황:
 1. offset 기능을 활성화한 경우, 모든 비교 결과가 전부 mismatch로 판정되는 문제
- 해결 과정:
 - 1) 비교 대상 데이터 간 값 차이가 항상 offset_value 만큼 일정하게 발생함을 로그를 통해 확인
 - 2) scoreboard에서는 offset이 적용되지 않은 raw 데이터와 DUT 출력 데이터를 직접 비교하고 있었음
→ 이로 인해 offset 사용 시 구조적으로 mismatch 발생
 - 3) addFunc.c 에 구현된 덧셈 연산 기능을 reference model로 활용하여 입력 데이터에 addFunc를 적용한 결과와 DUT 출력 값을 비교하도록 로직 변경
 - 4) 4번째 Trouble 발생. (Cont'd)

#5. C-Model 결과와 RTL 결과 비교 mismatch 발생

- 문제상황 :

1. C-Model 결과와 RTL 결과를 비교하는 과정에서 타이밍 불일치로 인한 mismatch 발생 문제

- 해결 과정:

- 1) o_de 신호를 통해 출력 타이밍을 제어하더라도 C Model의 출력 타이밍을 RTL과 동일하게 맞추는 것은 구조적으로 어려움.

→ 이로 인해 동일한 데이터임에도 불구하고 타이밍 불일치로 mismatch 발생.

- 2) 이때 타이밍을 강제로 맞추는 방식이 아니라, 비교 시점에서 latency를 제거하는 구조로 방향 전환

- 3) RTL 결과를 입력 시점에만 (i_de == H) queue 타입으로 저장하는 task와,

- 4) 출력 시점에 (o_de == H) queue에서 데이터를 pop 하여 비교하는 task 구현.

- 5) 이를 통해 출력 시점에 pop된 RTL 결과와 저장된 pixel 데이터 (C Model) 결과를 안정적으로 비교할 수 있도록 함.

EDA Playground Links

<https://edaplayground.com/x/icqs> -> pixel 단위로 저장

<https://edaplayground.com/x/dGEZ> -> line 단위로 저장

<https://edaplayground.com/x/pRV7> -> frame 단위로 저장



감사합니다.